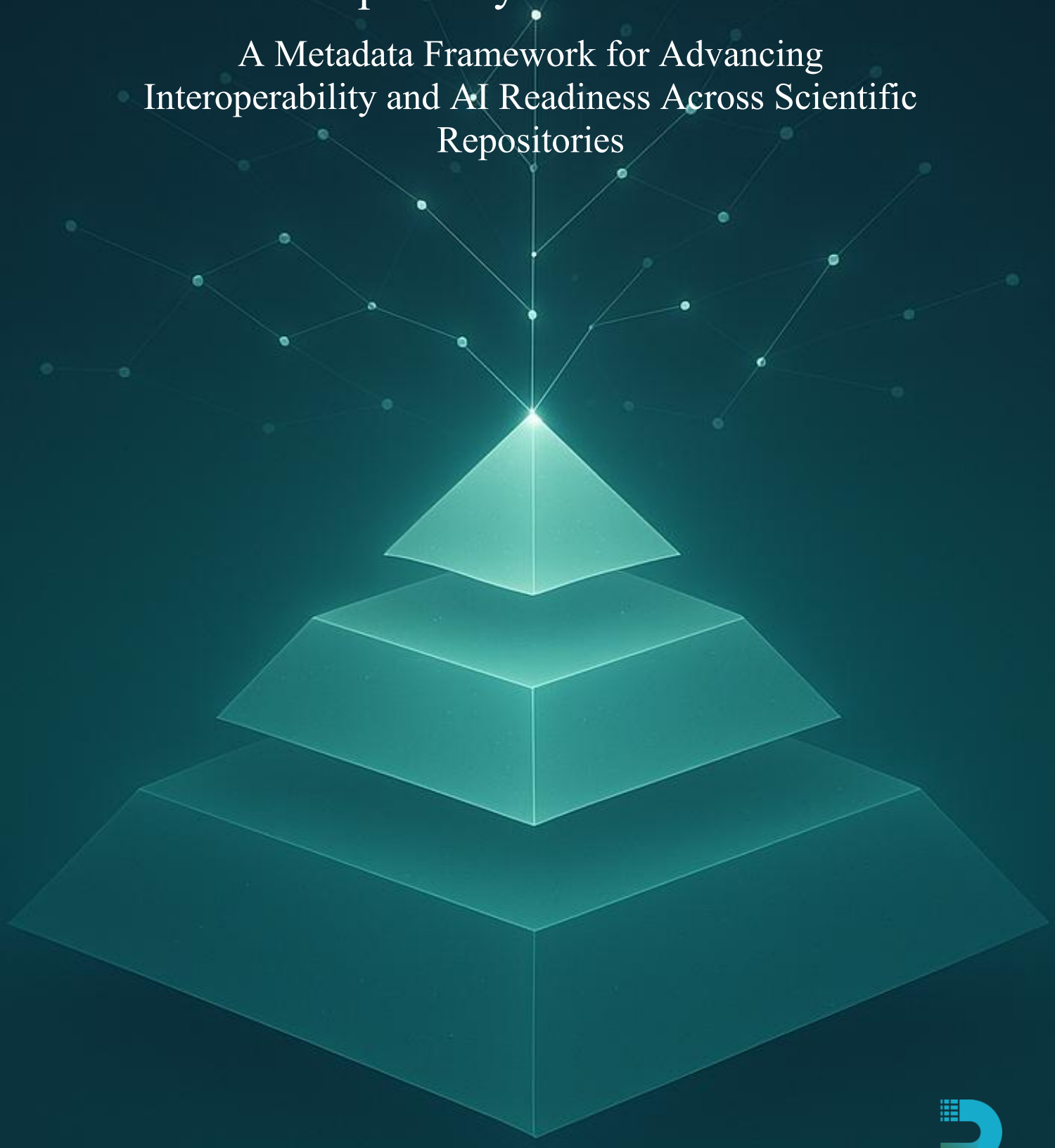
ORNL/TM-2026/4449

DOE Repository Metadata Profile

A Metadata Framework for Advancing Interoperability and AI Readiness Across Scientific Repositories



DOE Data Curation
Working Group

DOCUMENT AVAILABILITY

Online Access: US Department of Energy (DOE) reports produced after 1991 and a growing number of pre-1991 documents are available free via https://www.osti.gov.

The public may also search the National Technical Information Service's National Technical Reports Library (NTRL) for reports not available in digital format.

DOE and DOE contractors should contact DOE's Office of Scientific and Technical Information (OSTI) for reports not currently available in digital format:

US Department of Energy
Office of Scientific and Technical Information
PO Box 62
Oak Ridge, TN 37831-0062
Telephone: (865) 576-8401
Fax: (865) 576-5728
Email: reports@osti.gov
Website: www.osti.gov

Disclaimer. This research used resources of the Oak Ridge Leadership Computing Facility at the Oak Ridge National Laboratory, which is supported by the Office of Science of the U.S. Department of Energy under Contract No. DE-AC05-00OR22725.

This report was prepared as an account of work sponsored by agencies of the United States Government. Neither the United States Government nor any agency thereof, nor any of their employees, makes any warranty, express or implied, or assumes any legal liability or responsibility for the accuracy, completeness, or usefulness of any information, apparatus, product, or process disclosed, or represents that its use would not infringe privately owned rights. Reference herein to any specific commercial product, process, or service by trade name, trademark, manufacturer, or otherwise, does not necessarily constitute or imply its endorsement, recommendation, or favoring by the United States Government or any agency thereof. The views and opinions of authors expressed herein do not necessarily state or reflect those of the United States Government or any agency thereof.

License. This report is made available under a Creative Commons Attribution 4.0 International Public license (https://creativecommons.org/licenses/by/4.0).

ORNL/TM-2026/4449

DOE Repository Metadata Profile (DRMP)

A Metadata Framework for Advancing Interoperability and AI Readiness Across Scientific Repositories

Olga Anna Kuchar, Oak Ridge National Laboratory
Abigail Martin, National Energy Technology Laboratory
Brian J. Cain, Los Alamos National Laboratory
Kevin Kuhn, National Energy Technology Laboratory
Lauren Spath Luhring, National Laboratory of the Rockies
Juliane Schneider, Pacific Northwest National Laboratory
Jonathan Huitt, DOE Office of Scientific and Technical Information
Natalie Barnett, DOE Office of Scientific and Technical Information
Sara Studwell, DOE Office of Scientific and Technical Information
Miriam Blake, Pacific Northwest National Laboratory

Reviewed by the DOE Data Curation Working Group (DCWG)

November 11, 2025

Version 1.0

Table of Contents

Table of Contents *4*
Executive Summary *7*
1. Introduction *9*
1.1 Scope of this Document **10**
1.2 Intended Audience **10**
2. Background and Context *11*
2.1 Interoperability Foundations **11**
2.2 Mapping Logic and Structure **12**
2.3 Why Repository Metadata Matters **12**
2.4 Summary **13**
3. The DRMP Conformance Model *14*
3.1 Level 1 – Minimum Viable Record (MVR) **14**
3.2 Level 2 – Interoperability **15**
3.3 Level 3 – AI Readiness **17**
3.4 Progression and Validation **19**
3.4.1 Conformance Validation Workflow 19
3.5 Summary **20**
4. Integration Mechanisms: API and MCP Enablement *22*
4.1 API-Based Interoperability **22**
4.2 Semantic Context and the Model Context Protocol (MCP) **23**
4.3 Combined Workflow Example **25**
4.4 Implementation Guidance **26**
4.5 Relationship to DOE Systems **26**
4.6 Summary **27**
5. Schema Alignment and RDA-Informed Maturity Ratings *28*
5.1 The DRMP as a Schema-Alignment Framework **28**
5.2 RDA Attribute Gap Ratings **29**
5.3 Mapping Methodology **30**
5.4 Crosswalk Dimensions **30**
5.5 Representative Crosswalk Excerpt **31**

5.6 Interpretation and Rationale **32**

5.7 Relationship to Implementation (Chapter 6) **32**

6. Applying the DRMP Crosswalk: Implementation Guidance ***33***

6.1 Rosetta-Stone Matrix **33**

6.2 Implementing the DRMP Using Existing Metadata Schemas **34**

6.2.1 Identify Your Source Schemas 34

6.2.2 Map Internal Fields to DRMP Elements 34

6.2.3 Add Fields Needed for Level 2 or Level 3 Progression 34

6.3 Building a DRMP Record from Local Metadata **35**

6.4 Validating DRMP Records **35**

6.4.1 Schema Validation 36

6.4.2 Automated CI/CD Validation 36

6.4.3 MCP Validation (Level 3 Only) 36

6.5 Security and Access Metadata **36**

6.6 Implementation Checklist for Repository Teams **37**

6.7 Summary **39**

7. Governance and Change Management ***40***

7.1 Stewardship Model **40**

7.2 Versioning **40**

7.3 Deprecation Policy **40**

7.4 Extensions and Community Contributions **41**

7.5 Change Request Process **41**

7.6 Alignment with External Standards **41**

7.7 Long-Term Sustainability **42**

8. Conclusion ***43***

Appendices ***46***

Appendix A - DOE Repository Metadata Profile Schema ***47***

Appendix B — Crosswalk Matrix ***56***

Appendix C — OpenAPI and XML Specifications ***58***

C.1 Purpose and Scope **58**

C.2 Minimal OpenAPI 3.1 Example **58**

C.3 Minimal XML/XSD Representation Example **59**

C.3.1 XML Schema Definition (XSD) 59

C.3.2 Example XML Record 59

C.4 Implementation Guidance **60**

Appendix D — Model Context Protocol (MCP) Server Templates for AI Readiness ***61***

D.1 Design Overview **61**

D.2 Core Fields in MCP Manifest **61**
D.3 Example MCP Manifests **62**
D.4 Implementation Guidance **63**
D.5 Relationship to the DRMP Framework **63**
D.6 Summary **63**
Appendix D-1 — OpenAPI & MCP Endpoint Reference ***64***
D-1.1 Discovery & Manifests **64**
D-1.2 Standard OpenAPI Endpoints **64**
D-1.3 MCP Tool Conventions **66**
D-1.4 Versioning & Namespaces **66**
D-1.5 Authentication & Security Notes **67**
D-1.6 Error & Health Conventions **67**
D-1.7 Example URL Patterns (Canonical) **67**
Appendix D-2 – OpenAPI 3.1 Snippet for DMP Validation ***68***
Appendix E — DRMP Field Matrix ***73***
Core Repository Identification & Context **73**
Level 2 Interoperability – Technical & Policy Signals **73**
Level 3 AI-Readiness – Semantics, Preservation, and AI Governance **74**
Appendix F – Repository Policy Starter Kit ***76***
F.1 License Policy (Example Page) **77**
F-2 Retention Policy (Example Page) **78**
F-3 Harvest Policy (Example Page) **79**
F-4 Persistent Identifier (PID) Policy (Example Page) **80**
F-5 Preservation Policy (Example Page) **81**
F-6 AI Use Policy (Example Page) **82**

Executive Summary

The Department of Energy (DOE) funds a diverse and distributed ecosystem of repositories that steward scientific data, publications, and software across its research programs, user facilities, and national laboratories. While significant progress has been made in standardizing dataset-level metadata, the metadata describing repositories themselves (their identity, governance, access interfaces, policies, and technical capabilities) remains inconsistent and fragmented across DOE-funded systems. This variability limits discoverability, interoperability, automated validation, and AI-driven analysis, all of which are increasingly essential for modern scientific workflows. To address this gap, the DOE Data Curation Working Group (DCWG) developed the DOE Repository Metadata Profile (DRMP). The DRMP is a practical, community-driven framework that defines how repositories can describe themselves in a consistent, machine-actionable, and scalable manner.

The DRMP is not a new metadata schema. Instead, it is a **mapping profile and structured element set** capturing the essential characteristics of DOE repositories. It harmonizes repository-level metadata across six widely adopted community schemas: RE3Data; DCAT-US v3; Schema.org; Dublin Core; DataCite 4.6; and PREMIS 3.0. This harmonization eliminates reinvention and enables interoperability within DOE and across the broader scientific ecosystem. A core objective of the DRMP is to **reduce burden on repositories** by allowing them to reuse their existing metadata through a Rosetta-style crosswalk rather than redesigning local implementations.

The profile introduces a **three-level conformance model** that supports incremental adoption:

- **Level 1 – Minimum Viable Record (MVR):** foundational identification elements required for workflows, project registration, and basic repository presence.
- **Level 2 – Interoperable:** structured metadata enabling alignment with national and international discovery systems.
- **Level 3 – AI-Ready:** enhanced provenance, policy transparency, fixity, semantic context, and capabilities that support automated reasoning, model training governance, and machine-assisted curation.

To support implementation, the DRMP includes **JSON Schema definitions**, **OpenAPI patterns**, and **MCP templates** that allow repositories to publish machine-readable metadata directly within existing platforms. These resources are modular and lightweight, enabling adoption without major architectural change.

Adopting the DRMP enables repositories to:

- Enhance discoverability and interoperability by aligning identifiers, classifications, and descriptive elements across widely used schema standards.
- Support federated discovery and cross-registration across DOE systems, Data.gov, and international catalogs.

- Enable AI agents and workflow orchestration systems to interpret repository-level metadata within the American Science Cloud (AmSC) through Model Context Protocol (MCP)-based context publication.
- Demonstrate alignment with DOE's open science, stewardship, and FAIR data priorities.

This guidance represents a **community-driven step forward**. Through voluntary adoption and continued feedback, the DRMP advances a cohesive, machine-actionable description of DOE repositories that supports FAIR data practices, preparing the infrastructure for AI-enabled research, and strengthening the discoverability and reuse of DOE's scientific outputs.

1. Introduction

The Department of Energy (DOE) supports an expansive network of repositories that preserve, manage, and disseminate scientific data, software, and publications generated across its national laboratories, user facilities, and research programs. Although DOE repositories share common stewardship goals, they vary widely in purpose, scope, technical architecture, and metadata practices. This diversity reflects the scientific breadth of DOE's mission, but is also creates challenges for discovery, interoperability, automation, and AI-enables analysis.

Over the past decade, significant progress has been made in harmonizing *dataset-level metadata* through community schemas such as DataCite, Dublin Core, DCAT, and schema.org; however, the *metadata describing repositories themselves* remains unevenly implemented and inconsistently expressed across the DOE ecosystem. Repository metadata includes a repository's identity, governance, access interfaces, policies, and supported capabilities. This gap in repository metadata complicates efforts to integrate repositories into large-scale scientific workflows, federated catalogs, and the emerging American Science Cloud (AmSC). They also limit the ability of automated agents and AI systems to reason about repository trustworthiness, access pathways, and policy constraints.

The DOE Data Curation Working Group (DCWG) initiated the DOE Repository Metadata Profile (DRMP) to address these repository metadata gaps. The DRMP establishes a unified, machine-actionable set of repository-level descriptive elements that DOE repositories can adopt incrementally. It is not a new schema, but rather a structured, harmonized metadata profile that maps directly onto six widely adopted standards-RE3Data, DCAT-US v3, Schema.org, Dublin Core, DataCite 4.6, and PREMIS 3.0. By aligning with existing standards rather than replacing them, the DRMP reduces implementation burden while enabling repositories to reuse metadata they already maintain.

A key design goal of the DRMP is scalability; supporting lightweight adoption for small or project-level repositories, while providing richer semantic and policy detail for mature repositories integrating with AI and workflow automation. To support this, the DRMP introduces a three-tiered conformance model:

- Level 1 (Minimum Viable Record): establishes a repository's identity, ownership, and presence within DOE systems.
- Level 2 (Interoperable): enables alignment with national and international metadata catalogs through structured descriptions and declared policies.
- Level 3 (AI-Ready): provides the provenance, fixity, semantic context, AI governance metadata, and capability declarations needed for machine reasoning and AI-assisted workflows.

The DRMP also includes implementation resources: JSON Schema; OpenAPI patterns; and MCP-based context publishing templates. These resources will allow repositories to express profile elements in a standardized, machine-readable form without major system redesign.

Collectively, these efforts position the DRMP as a foundational component of DOE's broader open science, FAIR data, and AI-readiness initiatives. By adopting consistent repository-level

metadata, DOE repositories can increase discoverability, improve automation and cross-system integration, and strengthen the transparency and stewardship of scientific outputs.

1.1 Scope of this Document

This guidance focuses exclusively on *metadata about repositories themselves*, not the datasets, publications, or digital objects they host. The DOE Repository Metadata Profile (DRMP) defines an interoperability framework that:

- Harmonizes repository-level metadata across RE3Data, DCAT-US v3, Schema.org, Dublin Core, DataCite 4.6, and PREMIS 3.0.
- Provides examples of machine-actionable publication using JSON-LD, RDF, and OpenAPI-aligned patterns.
- Introduces a three-level conformance model supporting incremental repository adoption and validation.
- Supplies appendices containing crosswalk mappings, example payloads, policy templates, OpenAPI examples, and MCP patterns to support consistent implementation.

The DRMP does not define a new ontology or replace existing standards. Instead, it offers a harmonized mapping profile and implementation guide that transforms repository-level metadata from narrative description into machine-actionable infrastructure suitable for automation, workflow orchestration, and AI-enabled analysis.

1.2 Intended Audience

This document is intended for:

- Repository developers and technical professionals responsible for implementing metadata services, APIs, and integration workflows.
- Librarians, data curators, metadata specialists, and information scientists who design repository metadata models and ensure alignment with FAIR principles.
- DOE program managers, data stewards, and repository operators seeking a consistent framework for assessing repository readiness, interoperability, and compliance with federal policy.
- AI, ML, and workflow automation professionals who require structured repository descriptions to integrate repositories into automated pipelines, federated search platforms, and the AmSC.

The guidance assumes familiarity with common metadata standards (e.g., DCAT, Dublin Core, Schema.org) and provides a concise, actionable framework for aligning repository descriptions with DOE's open-science, stewardship, and AI-readiness goals.

2. Background and Context

The DRMP builds upon established community standards that already describe research repositories, catalogs, and preservation systems. Rather than defining a new schema, the DRMP *harmonizes* existing repository-level metadata across widely adopted standards to enable consistent, machine-actionable, and interoperable metadata within the DOE ecosystem. This approach reduces implementation burden, supports reuse of existing metadata, and provides a scalable pathway toward interoperability and AI readiness. The remainder of this chapter provides the background needed to understand the DRMP, including its interoperability foundations, mapping logic, and the role of repository-level metadata in DOE's open science and AI-readiness goals.

2.1 Interoperability Foundations

The DRMP draws on the complementary strengths of six mature metadata frameworks, each describing different aspects of repository identity, governance, access, and technical capabilities:

- RE3Data [4]: The authoritative international registry of research data repositories, defining attributes for repository identity, governance, funding, access rights, and technical interfaces.
- DCAT-US v3 [5]: The U.S. federal profile of the W3C Data Catalog Vocabulary, enabling repositories to expose structured metadata for integration with Data.gov and other federal discovery platforms.
- Schema.org [6]: A web-scale vocabulary widely used for structured metadata embedded in HTML pages and APIs, improving discoverability through commercial search engines and machine agents.
- Dublin Core [7]: A foundational general-purpose vocabulary for resource description, providing stable, widely recognized predicates for identity, rights, subject classification, and relationships.
- DataCite 4.6 [8]: A schema widely used for persistent identifiers and dataset-level metadata, offering compatible concepts for contributors, publishers, subjects, and funding bodies.
- PREMIS 3.0 [9]: A preservation metadata standard describing provenance events, object characteristics, fixity information, and preservation actions essential for long-term stewardship.

These six schemas form the basis of the DRMP Rosetta-style crosswalk presented in Appendix B:

- RE3Data provides the baseline descriptors.
- DCAT-US v3 enables federal interoperability.
- Schema.org improves web and agent discoverability.
- Dublin Core and DataCite offer general-purpose descriptive interoperability.
- PREMIS contributes provenance, fixity, and preservation semantics required for AI-ready curation.

Together, these mappings establish a layered, standards-aligned foundation supporting machine-actionable repository-level metadata across DOE and the broader scientific ecosystem.

2.2 Mapping Logic and Structure

The DRMP organizes repository-level metadata into conceptual groups that represent functional dimensions shared across the source schemas. These include:

- Identity and Classification
- Organizational and Funding Context
- Access, Interfaces, and Technical Capabilities
- Policies and Governance
- Provenance, Preservation, and Fixity
- Semantic Context and AI Governance

Within each dimension, DRMP fields are mapped to their nearest conceptual equivalents in the six source schemas. This harmonization accounts for naming differences, semantic equivalence, and machine-actionable structure. The mappings appear in:

- Appendix A: JSON Schema for machine-readable DRMP implementation
- Appendix B: Rosetta crosswalk mapping DRMP elements into six standards
- Appendix C: OpenAPI and XML examples for API-based metadata exchange
- Appendix F: Policy templates for Level 2 and Level 3 compliance

By following this structure, repositories can generate metadata that remains interoperable across RE3Data, DCAT-US, Schema.org, Dublin Core, DataCite, and PREMIS, while maintaining flexibility in local implementation. The profile also supports automated validation and self-assessment through the three conformance levels defined in Chapter 3: Minimum Viable Record (Level 1); Interoperable (Level 2); and AI-Ready (Level 3).

2.3 Why Repository Metadata Matters

Repository-level metadata is foundational to DOE's open science, FAIR data, and AI-readiness objectives. When expressed in a machine-actionable form, this metadata enables:

- **Transparency and trust:** Clear declarations of governance, policies, access models, and repository capabilities allow users and automated systems to evaluate repository trustworthiness and appropriateness for reuse.
- **Discoverability and interoperability:** Harmonized metadata ensures that DOE repositories can be integrated into RE3Data, Data.gov, FAIRsharing, and other national and international discovery ecosystems.
- **Automation and AI integration:** Structured metadata enables workflow systems, AI agents, and the American Science Cloud (AmSC) to identify repositories, understand their interfaces, and automate metadata harvesting, validation, and policy-aware reasoning.

- **Policy alignment and reporting:** Machine-actionable metadata supports compliance with DOE's Public Access Plan 2.0, FAIR metrics initiatives, and broader federal mandates for transparency and stewardship.

In this context, repository-level metadata operates as *infrastructure*, not documentation. By aligning DOE repositories around a common mapping profile and a scalable conformance model, the DRMP provides the foundation for automated, federated, and AI-assisted interoperability across DOE's scientific knowledge ecosystem.

2.4 Summary

Taken together, these interoperability foundations, mapping structures, and policy considerations establish the conceptual basis for the DRMP. They demonstrate why consistent, machine-actionable repository-level metadata is essential for FAIR practices, workflow integration, and AI-enabled research. This background provides the necessary context for the conformance model introduced in the next chapter.

3. The DRMP Conformance Model

The DRMP defines a three-level conformance model that provides repositories with a structured, progressive path toward machine-actionability, interoperability, and ultimately AI readiness. Each level introduces specific metadata, technical capabilities, and governance signals that enable repositories to function as identifiable, harvestable, and eventually AI-integrable components of DOE's scientific data ecosystem. The model is intentionally incremental: Level 1 establishes a minimum machine-actionable identity record; Level 2 adds the interfaces and policies required for automated harvesting and federated discovery; and Level 3 introduces the semantic alignment, access-control metadata (accessModel and authorizationModel), and AI-governance policies (such as aiUsePolicy) needed for agentic orchestration and participation in the AmSC.

Rather than enforcing a uniform standard, the DRMP recognizes that DOE repositories vary in maturity, design, and technical capacity. The conformance levels therefore offer an achievable, additive roadmap that repository teams can adopt over time, ensuring that each level supports clear operational, technical, and compliance outcomes.

The conformance model allows repositories to:

- Evaluate their current metadata coverage and technical interfaces against a standardized benchmark.
- Identify specific actions needed to progress toward full interoperability and semantic integration.
- Support operational readiness for automated harvesting, compliance monitoring, AI-governance signaling, and integration within agent-based environments.

Together, these levels form a roadmap that elevates repository metadata from static documentation to an extensible, machine-actionable infrastructure that supports discovery, compliance, and cross-domain integration. This chapter presents each conformance level in detail, outlining the required elements, technical capabilities, and validation steps that constitute the DRMP maturity model.

3.1 Level 1 – Minimum Viable Record (MVR)

Objective: Establish a minimal, machine-actionable identity record that allows a repository to be uniquely referenced, validated, and registered within DOE data ecosystems.

Level 1 defines the baseline metadata sufficient for a repository to exist as a uniquely identifiable, citable, and trackable component of the DOE scientific enterprise. It intentionally excludes descriptive, policy, API, or semantic metadata. Instead, if focuses on essential identity attributes require to:

- Anchor the repository to a global or local persistent identifier.
- Identify the operating organization.
- Provide a minimal trust and contact context.
- Allow automated systems to ingest, route, and validate the repository's identity.

A Level 1 record contains exactly five required fields, plus a required `conformsTo` declaration:

1. `repositoryIdentifier` – a persistent, globally unique identifier.
2. `repositoryName` – a human-readable name (e.g., ORNL Constellation).
3. `repositoryType` – a controlled vocabulary term describing the repository category (e.g., institutional, disciplinary).
4. `hostingInstitution` – the responsible organization.
5. `contactPoint` – a support or administrative email address.
6. `conformsTo` – must equal "Level-1-MVR".

These fields form the *minimum machine-actionable footprint*. They allow any DOE system such as registries, workflow engines, metadata validators, or AmSC components to recognize the repository, validate its existence, and anchor additional metadata layers.

Optional fields such as `fundingBody` or `repositoryDescription` may be included but are not required at Level 1.

Level 1 therefore represents the "registration[1] tier" where a repository can be indexed, cited, and managed within DOE infrastructure, but is not yet interoperable or semantically enriched.

3.2 Level 2 – Interoperability

Objective: Provide machine-readable interfaces, technical metadata formats, and operational policies required for federated discovery, automated harvesting, and cross-enclave integration.

Level 2 introduces interoperability capabilities that move a repository beyond simple registrability and into DOE-wide discovery and data lifecycle ecosystem. At this level, metadata exposure becomes machine-actionable, enabling automated harvesting, validation, monitoring, and version-aware ingestion.

A Level 2 repository must implement the following required metadata elements, in addition to all Level 1 fields:

API Exposure and Formats

- `apiEndpoints` – network-accessible machine interfaces (e.g., REST, JSON, OAI-PMH).
- `metadataFormats` – describes the repository-level metadata formats or schemas that the respository exposes through its interfaces and exports, not the per-dataset internal metadata formats (e.g., Dublin Core, DCAT-US).

[1] Here, "registered" refers to establishing a machine-actionable identity record within DOE-managed metadata systems, not external registration in services such as RE3Data or FAIRsharing.

These fields allow aggregators, search services, and automated agents to retrieve repository and dataset-level metadata in predictable formats.

Policy Surfaces for Automation

- `licensePolicy` – a machine-readable URI referencing terms of use or licensing conditions.
- `retentionPolicy` – a machine-readable URI referencing retention or disposition documentation.
- `harvestPolicy` – a machine-readable URI describing harvesting permissions.

Declaring these policies enables compliant, automated harvesting across DOE and external systems such as Data.gov, FAIRsharing, RE3Data, OpenAIRE (Guidelines 4.0), and European Open Science Cloud (EOSC).

Temporal Versioning and Trust Elements

- `issued` – timestamp when the repository description was created.
- `modified` – timestamp for the most recent update.
- `stewardshipCommitment` – a machine-readable statement of the repository's curation scope (short-term access only, medium-term curation, or long-term preservation)

These allow monitoring tools and metadata pipelines to detect changes, generate deltas, and track repository evolution over time.

Technical Capabilities

- `persistentIdentifierPolicy` – declares which PID systems the repository routinely uses (`pidSystems`)and may optionally point to a human-readable, versioned PID governance policy (`pidPolicy`)
- `capabilities` – machine-actionable declaration of supported repository features, including Level 2–relevant operations such as:
    - `apiAccess`
    - `bulkDownload`
    - `datasetVersioning`
    - `webhooks`
    - `schemaBrowse`

These enable DOE systems to reason about whether a repository supports the operational requirements of integration pipelines (e.g., whether it supports dataset-level versioning for incremental harvesting).

A Level 2 repository must set "`Level-2-Interoperable`" within the `conformsTo` field.

At this level, repositories become *interoperable across systems*. Their metadata can be automatically harvested, validated, and federated, enabling participation in:

- DOE facility catalog services

- Data.gov (DCAT-US v3) pipelines
- FAIRsharing and RE3Data crosswalks
- cross-enclave metadata brokers
- knowledge graph construction
- schema.org–indexed search environments

Optional but recommended interoperability metadata:

- `accessModel` - open, authenticated, or restricted
- `authorizationModel` - none, API key, OAuth2, federated

Level 2 is the *operational bridge* between a repository's local implementation and the broader interconnected infrastructure of the DOE scientific enterprise. It becomes a technically interoperable node within DOE where its metadata can be harvested, parsed, validated, and federated by automated systems.

3.3 Level 3 – AI Readiness

Objective: Enable repositories to participate in AI-driven workflows by exposing semantic, provenance, authorization, and policy metadata required for automated reasoning, trust scoring, and agentic orchestration.

Level 3 represents the highest maturity tier in the DRMP conformance model. At this level, repositories publish the rich machine-actionable information that allow AI systems, reasoning engines, and MCP-based agents to safely interact with the repository.

A Level 3 repository must implement all Level 1 and Level 2 requirements, plus the following AI-readiness elements:

Semantic Grounding

- `ontologyAlignment` – URIs of ontologies, controlled vocabularies, or schemas used by the repository. Implementers may also include versioned URIs in `ontologyVersion` corresponding to those ontologies where available.
- `preservationPolicy` – a machine-readable statement of long-term preservation commitment.
- semanticContext – JSON-LD or RDF @context URIs that provide machine-readable mappings from DRMP and local fields to ontology or schema URIs. Repositories that implement MCP **should** include their MCP context URL (e.g., `/.well-known/mcp/context.jsonld`) in this list.

These enable semantic enrichment, knowledge graph alignment, and trust-aware AI systems.

Provenance and Integrity

- `provenance` – PREMIS event records describing key repository actions (validation, transformation, updates).
- `fixity` – PREMIS fixity metadata to enable integrity verification.

These support confidence scores and automated trust evaluations.

AI Use and Governance

- `aiUsePolicy` – machine-readable policy controlling AI/ML use of the repository's content or metadata (training allowed, derivative models allowed, commercial use allowed). Repositories may also provide an `aiPolicy` URI that links to a human-readable AI use or AI governance policy document.

This ensures that AI agents respect legal, ethical, and contractual constraints. Also note that since the DRMP is a repository-level profile, the `aiUsePolicy` sub-elements describe the repository's default stance. For general-purpose repositories where permissions vary by dataset, sub-elements may use the `dataset-specific` value to indicate that AI tools must consult dataset-level rights for precise guidance.

Access and Authorization Models

- `accessModel` — required for ML agents determining usability
- `authorizationModel` — allows AI agents to autonomously request tokens

These allow AI agents to determine whether they can access repository APIs and how to authenticate safely.

MCP Tools and Agent Interfaces

- `mcpTools` – machine-readable MCP tool descriptors exposing repository metadata to AI agents and orchestration systems.

This is the critical interface enabling "describe," "query," and "inspect" operations in the Model Context Protocol (MCP) ecosystem.

A Level 3 repository must set "`conformsTo`": "`Level-3-AIReady`".

At this level, repositories become *AI-ready nodes* capable of supporting:

- Autonomous metadata discovery
- Agentic curation workflows
- Trust evaluations and safety constraints
- Knowledge graph construction
- Semantic routing
- AI-driven compliance reporting

Level 3 repositories forms the backbone of the AmSC's machine-actionable metadata ecosystem.

3.4 Progression and Validation

The DRMP conformance model is incremental. Repositories may begin at Level 1 and progressively adopt Level 2 and Level 3 capabilities as they mature. Validation is performed using the DRMP JSON Schema described in Appendix A, which enforces level-specific requirements through conditional schema logic.

Validation steps include:

- Schema validation for required fields, type correctness, and controlled vocabularies.
- API endpoint validation (Level 2 and Level 3): checking endpoint availability, metadata formats, error handling, and declared capabilities.
- Policy validation: ensuring `harvestPolicy`, `licensePolicy`, `retentionPolicy`, and `aiUsePolicy` are machine-readable URIs.
- Access/authorization model evaluation to confirm that Level 3 repositories clearly declare authentication constraints.
- MCP tool validation for Level 3 repositories to ensure all MCP descriptors are complete and well-formed.
- Provenance and fixity validation for PREMIS elements.
- Generation of machine-actionable validation reports that may be published or integrated into DOE compliance workflows.

Conformance levels are additive; achieving Level 3 implies full compliance with Level 1 and Level 2.

3.4.1 Conformance Validation Workflow

To support consistent adoption, the DRMP defines a unified validation workflow applicable to all conformance levels. Validation may be performed by repository teams, automated CI pipelines, or external assessment tools.

Step 1 — Assemble a Repository-Level Description
Prepare a DRMP record using the JSON Schema in Appendix A. The record may be generated manually or exported from an internal metadata system.

Step 2 — Select a Target Conformance Level
Declare the desired conformance level using `conformsTo` element.

Step 3 — Validate Against the DRMP JSON Schema
Use the DRMP JSON Schema validator (Appendix A) to confirm:

- required elements for the selected level
- field types and controlled vocabularies
- deprecation warnings
- RDA anchor consistency (where applicable)

Step 4 — Validate API or MCP Capabilities (Level 2 or Level 3 Only)
For higher levels:

- Level 2: Validate required API endpoints, response structures, pagination behavior, and error handling (Appendix C, Chapter 4).
- Level 3: Validate MCP tool definitions, context-publication patterns, and semantic alignment (Appendix D, Chapter 4).
- Verify that required capabilities are declared and align with the repository's exposed interfaces.
- For Level 3: validate aiUsePolicy, accessModel, authorizationModel, and the structure of mcpTools definition.

Step 5 — Generate a Machine-Actionable Validation Report
The DRMP validation tool outputs a structured JSON document such as:

```
{
  "ok": true,
  "level": "L2",
  "errors": [],
  "warnings": ["licensePolicy recommended for Level 2"],
  "schema": "https://repo.example.org/schema/drmp-v1.0.json"
}
```

Step 6 — Register Outcome (Optional)
Repositories may publish the validation result:

- within the repository website
- within an MCP `describe` tool output
- to future DOE registry services (as they become available)

3.5 Summary

The DRMP conformance model offers a scalable, machine-actionable pathway for repositories to advance from basic registration (Level 1) to technical interoperability (Level 2) and, ultimately, AI readiness (Level 3). Each level adds specific machine-actionable capabilities, policies, and semantic features aligned with DOE requirements and community best practices.

Level 2 enables repositories to participate in automated harvesting, metadata federation, and cross-enclave workflows. Level 3 introduces the semantic, governance, and agentic interfaces needed for AI-driven operations, trust scoring, and participation in the MCP ecosystem. Levels 2 and 3 additionally introduce machine-actionable policy, capability, and AI-governance metadata (e.g., harvestPolicy, capabilities, aiUsePolicy, and MCP tool declarations) which collectively enable full interoperability and AI readiness.

By following this model, repositories transition from isolated metadata silos into dynamic, machine-actionable components of DOE's scientific and AI infrastructure, forming the metadata

foundation for the AmSC and future autonomous research environments. This progression lays the foundation for the integration mechanisms and metadata workflows detailed in Chapter 4.

4. Integration Mechanisms: API and MCP Enablement

The DRMP's conformance model becomes operational through a set of integration mechanisms that translate metadata standards into actionable interoperability. Where Chapter 3 defines *what* repositories must expose at each maturity level, this chapter describes *how* those elements are used to enable machine-actionable discovery, harvesting, semantic exchange, and AI-driven orchestration. At Level 2, repositories surface structured metadata through standard APIs along with capability declarations, harvest policies, and machine-readable governance signals. At Level 3, repositories publish semantic context and agent-interaction descriptors using the Model Context Protocol (MCP), enabling AI agents in the AmSC to reason over repository policies, capabilities, and trust attributes.

Together, these mechanisms ensure that repositories do not operate as isolated metadata silos, but as interoperable, machine-negotiable components of the DOE scientific data ecosystem.

This chapter details how repositories implement these capabilities in practice, describing the API conventions required for Level 2 interoperability and the MCP-based semantic context mechanisms required for Level 3 AI readiness.

4.1 API-Based Interoperability

APIs provide the first operational layer of interoperability under the DRMP. They allow repositories to expose metadata in structured, machine-readable formats suitable for automated harvesting, validation, monitoring, and integration into discovery platforms.

For repositories achieving Level 2 conformance, API-based interoperability requires the following practices and supporting metadata:

Core API Principles
Repositories SHOULD:

- Provide REST or OAI-PMH endpoints for repository-level (and dataset-level) metadata.
- Support JSON, JSON-LD, or RDF serializations, enabling semantic clarity and transformation.
- Align API responses with DCAT-US v3, Schema.org Dataset, DataCatalog, and Organization classes.
- Propagate persistent identifiers (PIDs) such as DOIs, ARKs, and ROR IDs at all appropriate entity levels.
- Publish API documentation using OpenAPI 3.1, enabling automated validation by external agents.
- Ensure all endpoints support consistent pagination, rate limiting, and error reporting conventions.
- Required Level 2 Metadata for API Interoperability

In addition to core DRMP Level 2 fields, the schema requires several elements that directly support API interaction:

- `apiEndpoints` - Machine-readable URIs exposing metadata.
- `metadataFormats` - Serialization formats supported by the API.
- `licensePolicy` - Machine-readable license or terms-of-use URL.
- `retentionPolicy` - Documented retention and disposition policy.
- `harvestPolicy` - A required declaration specifying whether and how external systems may legally or operationally harvest metadata.
- `stewardshipCommitment` - A repository's declared scope of curation or preservation.
- capabilities - Machine-readable declarations of supported technical operations, such as:
    - `apiAccess`
    - `bulkDownload`
    - `datasetVersioning`
    - `webhooks`
    - `schemaBrowse`

Operational Impact
These fields enable interoperable behavior across DOE and public registries:

- Data.gov, RE3Data, FAIRsharing, and OSTI can harvest, validate, and classify repository metadata automatically.
- DRMP-aligned APIs become inputs to DOE data-lifecycle automation, compliance pipelines, and observability dashboards.
- Automated validation frameworks (Appendix A) can test endpoint completeness, policy exposure, and conformance.

API-based interoperability establishes a robust foundation on which Level 3 semantic and AI-facing capabilities can be built.

4.2 Semantic Context and the Model Context Protocol (MCP)

While APIs provide access to metadata records, MCP enables exchange of context, meaning structured information describing how repositories manage, expose, and interrelate their resources. MCP defines a lightweight, machine-interpretable framework that allows repositories to communicate their operational capabilities, interfaces, and governance logic directly to AI agents and workflow systems.

In the DRMP, MCP integration is required for Level 3 conformance. The following metadata and descriptors must be available in machine-actionable form:

Required Level 3 Metadata Fields

- `ontologyAlignment` - Links to domain ontologies or controlled vocabularies relevant to repository content.
- `preservationPolicy` - A machine-readable preservation commitment.
- `provenance` - PREMIS event metadata describing validation, updates, and operational change history.
- `fixity` - Checksums enabling integrity verification of metadata representations.

- `accessModel` - Declares whether APIs are `open`, `authenticated`, or `restricted`.
- `authorizationModel` - Specifies mechanism (`none`, `api-key`, `oauth2`, or `federated`).
- `aiUsePolicy` - Machine-readable AI-governance metadata including training, derivative models, commercial use, and other constraints.
- `capabilities` - Expanded list for AI-facing systems, including:
  - `semanticSearch`
  - `vectorSearch`
  - `embeddings`
  - `reasoningSupport`
  - `mcpTools`

Distinguishing "capabilities" from "mcpTools"
The DRMP intentionally separates declaring support for MCP from describing the MCP tools themselves:

- `capabilities` includes a lightweight flag (e.g., "mcpTools") indicating that MCP integration is available. This allows AI agents to triage repositories quickly.
- `mcpTools` contains the actual MCP tool descriptors that are detailed, structured definitions of operations that an agent may invoke.

A Level 3 repository must therefore expose:

- `mcpTools` - A list of MCP tool descriptors that define agent-accessible operations (e.g., search, describe, validate, resolve).

Example MCP tool descriptor:

```
{
  "toolName": "list-datasets",
  "endpoint": "https://repo.example.org/mcp/list-datasets",
  "description": "Returns an array of dataset identifiers available in the
repository."
}
```

MCP in AI-enabled ecosystems
Through these capabilities and descriptors, MCP enables:

- autonomous selection of compatible repositories
- reasoning over access rules, provenance integrity, and ontology alignment
- integration with AI pipelines that require trust, validation, and policy signals
- dynamic orchestration within the AmSC

By exposing MCP context manifests and tool definitions, repositories become first-class participants in AI workflows, capable not only of being harvested, but of being queried, validated, and utilized by autonomous systems.

4.3 Combined Workflow Example

A repository implementing all DRMP levels typically participates in the following interaction flow:

1. Level 1 - Registration
The repository exposes essential identity metadata:

- persistent identifier
- hosting institution
- contact point
- repository type

This enables initial registration and discoverability across DOE systems and external registries.

2. Level 2 - Interoperability
The repository publishes machine-readable interfaces and policies, including:

- API endpoints (REST or OAI-PMH)
- supported metadata formats
- harvestPolicy declaring harvesting permissions
- license and retention policies
- Level 2 capabilities (e.g., apiAccess, bulkDownload, versioning)

These enable harvesters (e.g., Data.gov, OSTI, FAIRsharing) to automatically ingest, transform, and validate metadata with no manual intervention.

3. Level 3 - AI Readiness
The repository publishes an MCP context manifest containing:

- AI governance metadata (aiUsePolicy)
- accessModel and authorizationModel
- provenance and fixity metadata
- ontologyAlignment and preservationPolicy
- Level 3 capabilities (e.g., semanticSearch, vectorSearch, embeddings)
- MCP tool descriptors (mcpTools) describing agent-accessible operations

This enables AI agents to understand not only what the repository exposes, but also how it must be accessed and how its metadata may be used within AI-driven workflows.

4. Automated Harvest + Context Consumption (AmSC)
The AmSC or DOE AI agents:

- harvest the Level 2 API
- validate conformance against the DRMP JSON Schema
- ingest and interpret the MCP manifest
- assess training permissions, access controls, capabilities, and provenance signals

This produces a complete, machine-interpretable view of repository readiness and operational characteristics.

5. AI-Orchestrated Integration
Agents then federate repository resources into:

- data pipelines
- validation workflows
- knowledge graphs
- cross-repository analytical environments

Agents automatically select compatible repositories based on semantic context, declared capabilities, and AI governance metadata.

In conclusion, this layered approach transforms a repository from a static metadata source into a fully governed, semantically enriched, AI-integrable resource.

4.4 Implementation Guidance

To support consistent adoption across DOE repositories, implementers should:

- Embed DRMP field names directly in API responses or MCP manifests to facilitate automated mapping.
- Use persistent identifiers (e.g., ORCID, ROR, DOI) wherever possible to ensure entity disambiguation.
- Automate DRMP validation using tools described in Appendix A before public release.
- Version and document API schemas, MCP manifests, and capability declarations to maintain backward compatibility for downstream services.
- Publish API and MCP endpoints in human-readable landing pages and machine-readable well-known locations.
- Use semantic web standards (e.g., JSON-LD, RDF) to align repository metadata with global discoverability guidelines.

These operational practices ensure that repositories can reliably progress from basic registration to full AI readiness.

4.5 Relationship to DOE Systems

DRMP integration mechanisms align repositories with broader DOE and federal discovery infrastructure, including:

- OSTI: DRMP-aligned metadata can be transmitted directly for archiving and reporting.
- DOE Data Explorer: DRMP APIs ensure seamless ingestion without custom transformation layers.
- Data.gov / Catalog.data.gov: DCAT-US v3 alignment enables automatic harvesting and synchronization.
- FAIRsharing / RE3Data: Metadata and capability alignment supports classification in community registries.

- AmSC: MCP manifests enable AI agents to automatically onboard repositories, assess availability, and orchestrate workflows using repository capabilities and policies.

This interoperability places DOE repositories within both national and international open-science ecosystems.

4.6 Summary

APIs and MCP form complementary layers within the DRMP integration framework. API-level interoperability provides structured, harvestable metadata aligned with federal and community standards. MCP-level semantic context adds governance, provenance, capability declarations, and AI-specific operational metadata that enable autonomous agent interaction. Together, these mechanisms elevate repositories from passive catalogs to active, intelligent participants in DOE's emerging AI-enabled research infrastructure and set the stage for the crosswalk and rationale presented in Chapter 5.

5. Schema Alignment and RDA-Informed Maturity Ratings

As repositories evolve into core components of DOE's scientific data ecosystem, a consistent metadata language becomes essential for discoverability, interoperability, and trust. This chapter describes how the DRMP aligns multiple international and federal metadata standards into a single, machine-actionable framework for describing repositories. The DRMP crosswalk forms the operational backbone of this framework: it harmonizes established schemas so that repository-level metadata across DOE systems is both interoperable and progressively AI-ready. Beginning at Level 2, the alignment incorporates machine-actionable access and authorization metadata; by Level 3, it integrates AI-governance and capability signals required for agent-driven workflows.

The DRMP also draws on the Research Data Alliance (RDA) Common Descriptive Attributes, using their Gap 1-3 maturity ratings as a conceptual lens for assessing metadata completeness and effort. These ratings help quantify the progression described in Chapter 3, but the DRMP does not map directly to RDA fields; instead, RDA informs how maturity expectations are structured and communicated.

The DRMP therefore consists of two complementary components:

- The *conformance model*, formalized in the JSON Schema in Appendix A, which defines the metadata, technical capabilities, and governance expectations for Levels 1 through 3; and
- The *schema crosswalk*, provided in Appendix D, which aligns each DRMP element with corresponding representations across six community schemas (RE3Data, DCAT-US v3, Schema.org, Dublin Core, DataCite, PREMIS).

Together, these components provide a coherent framework for aligning DOE repository metadata with community standards, enabling interoperability, policy compliance, and AI readiness across the DOE scientific knowledge ecosystem.

5.1 The DRMP as a Schema-Alignment Framework

The DRMP defines a tiered maturity model for repository-level metadata. Each conformance level functions both as a maturity stage and a schema-alignment layer, enabling repositories to strengthen interoperability and machine-actionability while remaining backward-compatible with prior levels.

At its core, the DRMP serves as a **profile-mapping framework**—a *Rosetta Stone* that aligns metadata elements across six major community schemas:

Schema	Version	Purpose
RE3Data	current	Baseline for global repository discovery
DCAT-US v3	2023 (U.S. profile of W3C DCAT 3.0)	Federal interoperability and open-data compliance

Schema.org	continuous	Web-scale indexing and discoverability
Dublin Core (DCMI 1.1)	2020	Cross-domain descriptive foundation
DataCite 4.6	2023	DOI registration and dataset citation
PREMIS 3.0	2021	Preservation, provenance, and lifecycle integrity

DRMP Level 3 extends this alignment to include AI-governance metadata (aiUsePolicy), machine interface characteristics (capabilities), and MCP tool descriptors, enabling repositories to participate in autonomous AI systems.

Together these schemas span the full descriptive spectrum of repository metadata, from identity and discovery (RE3Data, DCAT, Schema.org, Dublin Core) to persistence and provenance (DataCite, PREMIS).

The DRMP crosswalk harmonizes overlapping attributes into a single structured model that can be serialized in JSON-LD, RDF, or XML, ensuring that repository records are equally interpretable by humans, search engines, and AI systems.

5.2 RDA Attribute Gap Ratings

The DRMP incorporates the Research Data Alliance (RDA) *Common Descriptive Attributes of Research Data Repositories* (2023) and their associated Gap 1–3 maturity ratings, providing a quantitative mechanism for assessing metadata completeness and measuring repository progression through the conformance levels described in Chapter 3. While the DRMP does not attempt to replicate the full RDA crosswalk, it adopts the Gap 1–3 maturity framework as a grounding reference for assessing the availability and sophistication of repository-level metadata.

Gap Level	Definition	Typical Expectation	DRMP Treatment
Gap 1 – Readily Available	Attribute is typically present and easy to locate in existing repository records.	Repository name, identifier, URL, contact, organization.	**Mandatory** for Level 1 (Minimum Viable Record) compliance.
Gap 2 – Partially Available	Attribute is inconsistently exposed or difficult to locate across repositories.	Metadata standard, API endpoint, license, access policy.	**Required** for *Level 2 (Interoperable)* to strengthen cross-catalog integration.
Gap 3 – Rare or Advanced	Attribute is seldom available or requires specialized infrastructure.	Provenance, ontology alignment, fixity, preservation policy.	**Recommended / Required** for *Level 3 (AI-Ready)* repositories.

Interpretation:
Gap ratings correspond to a repository's metadata-transparency maturity:

- Gap 1 → Discoverability foundation
- Gap 2 → Interoperability expansion

- Gap 3 → Machine-actionable readiness and trustworthiness

Each DRMP checklist entry retains its original RDA gap score so implementers can gauge the effort required to meet higher conformance levels.

These ratings also inform DOE-wide dashboards for FAIR compliance and AI-readiness tracking, creating a common quantitative bridge between individual repository progress and cross-program performance metrics.

5.3 Mapping Methodology

The crosswalk alignment followed a structured methodology based on semantic equivalence, machine-actionability, and policy relevance.

Semantic Equivalence Mapping
Core fields (e.g., repositoryIdentifier, repositoryName, hostingInstitution) were mapped across RE3Data, DCAT, Schema.org, Dublin Core, and DataCite. When multiple representations existed, the DRMP selected the form offering the highest machine readability, typically the DCAT or Schema.org JSON-LD expression.

Normalization and Controlled Vocabularies
DRMP Level 3 extends this alignment to include AI-governance metadata (aiUsePolicy), machine interface characteristics (capabilities), and MCP tool descriptors, enabling repositories to participate in autonomous AI systems.

Controlled lists draw from community registries such as ROR for institutions and SPDX for licenses.

Layered Integration with PREMIS
For repositories that manage long-term holdings, DRMP fields extend to preservation and provenance elements using PREMIS 3.0 concepts (object, event, agent, rights).

This alignment ensures that DOE repositories can express preservation integrity alongside accessibility and interoperability.

AI Governance Alignment
Level 3 mappings incorporate AI governance metadata through aiUsePolicy, which aligns conceptually with rights metadata in Dublin Core and DataCite, and with risk/governance assertions emerging in FAIRsharing and schema.org/CreativeWork extensions.

5.4 Crosswalk Dimensions

The DRMP organizes repository-level metadata into five logical dimensions that map directly to the six community schemas:

- Identity and Classification — persistent identifiers, official and alternate names, repository type.
- Organizational Context — hosting institution, funding body, stewardship commitment, ROR integration.

- Technical and Access Characteristics — supported data formats, APIs, authentication methods, PID systems.
- Policy and Governance — licensing, retention, preservation policy, rights and terms of use.
- AI Readiness and Machine-Actionability — ontology alignment, AI use policies, access and authorization metadata, semantic and vector capabilities, MCP tool descriptors, and contextual metadata required for agentic integration.

Each dimension groups attributes that share a common function and can be validated together. Appendix D provides the full mapping table linking each DRMP field to its source schema(s), enumerations, and RDA Gap rating.

5.5 Representative Crosswalk Excerpt

To demonstrate the practical application of the DRMP, the table below presents a representative portion of the crosswalk matrix. It shows how key repository attributes map across multiple community schemas and how corresponding RDA Gap levels convey metadata readiness within each dimension.

Dimension	Field	Source Schema(s)	Description	Gap
Identity & Classificat-ion	repositoryIdentifier	RE3Data → dct:identifier; DCAT → dct:identifier	Persistent identifier (DOI or OSTI ID) for repository	1
Organizat-ional Context	hostingInstitution	RE3Data: institution; Schema.org: provider; DataCite: publisher	Hosting organization (ROR preferred)	1
Technical & Access	apiEndpoint	DCAT:dcat: endpointURL; Schema.org: contentUrl	Base URL for machine-readable metadata (JSON-LD/RDF)	2
Policy & Governance	licensePolicy	Dublin Core: dct:license; DataCite: rightsURI	Declared license URI (e.g., CC BY 4.0)	2
AI Readiness	ontologyAlignment	Schema.org: about; PREMIS: objectCharacteristics; MCP context	Declared ontology or schema alignment for AI integration	3

Technical & Access	accessModel	DCAT: accessRights; Schema.org: conditionsOfAccess	High-level access model for APIs	2
AI Readiness	aiUsePolicy	Dublin Core: rights; DataCite: rightsURI; MCP context	Machine-readable policy describing whether metadata/content may be used for AI training or derivation	3

5.6 Interpretation and Rationale

The crosswalk enables semantic continuity across repository ecosystems while supporting incremental adoption. By linking DOE metadata to the six community schemas and scoring each field with RDA Gap ratings, the DRMP provides:

- Consistent assessment of metadata quality and completeness.
- Machine-actionable infrastructure that supports automated discovery, harvesting, and policy compliance.
- A scalable pathway for repositories to evolve from human-readable documentation to AI-interpretable context.
- Direct policy traceability to OBBB (§50404 AmSC), DOE Public Access Plan 2.0, and FAIR data mandates.

Higher-Gap attributes represent the transition from descriptive metadata to functional, machine-actionable infrastructure, including AI-governance signals, operational capabilities, and MCP descriptors, that enable AI agents to interpret repository context, constraints, and compatibility.

Their implementation forms the foundation for Model Context Protocol (MCP) interactions described in Chapter 4.

5.7 Relationship to Implementation (Chapter 6)

This chapter has defined *what* to align and *why*, such as the schema mappings, gap logic, and rationale underpinning the DRMP. **The following chapter focuses on implementation**, detailing how repositories publish, validate, and sustain these alignments in practice through standardized publication formats, validation pipelines, and adoption mechanisms that translate this crosswalk into operational reality across DOE repositories. Implementation also requires publishing AI governance metadata, access-control declarations, capabilities, and MCP tool descriptors so that automated agents and the AmSC can evaluate repository readiness, provenance, and policy constraints.

6. Applying the DRMP Crosswalk: Implementation Guidance

To recap, Chapters 3 - 5 established the conceptual foundations of the DRMP by defining its conformance levels, describing the API and MCP integration mechanisms, and outlining the schema-alignment strategy. This chapter focuses on *practical adoption*. It describes how repository operators can implement the DRMP using metadata they already maintain, map existing schemas onto the profile, and validate their repository-level records through machine-actionable tools.

Rather than prescribing a new metadata system, the DRMP is designed to *wrap around and harmonize* existing repository descriptions. Most DOE repositories already expose metadata using RE3Data fields, DCAT distributions, Dublin Core tags, DataCite elements, or custom schemas embedded in Exchange, CKAN, Dataverse, IDEAS, or homegrown repositories.

Chapter 6 therefore demonstrates how the DRMP can be implemented *without restructuring internal systems*, by using the crosswalk mappings and validation scaffolding provided in this chapter and in Appendices A and B. We introduce the full Rosetta-Stone crosswalk matrix, describe an implementation process for generating DRMP records from existing metadata, discuss security and access expression, and finish with a practical implementation checklist.

6.1 Rosetta-Stone Matrix

Chapter 5 introduced the standards underlying the DRMP, which are RE3Data, DCAT-US v3, Schema.org, Dublin Core, DataCite 4.6, PREMIS 3.0, and the RDA Common Descriptive Attributes; it only provided a snippet, but did not present the full crosswalk. This section provides that crosswalk in operational form.

The Rosetta-Stone Matrix is the central implementation tool of the DRMP. It aligns each DRMP metadata element with its equivalent representations across the six community schemas and specifies the conformance level (Level 1, 2, or 3) at which the element becomes required.

Rather than functioning as a new schema, the matrix demonstrates that implementers rarely need to restructure existing metadata systems. Most repositories already maintain functionally equivalent fields under RE3Data, DCAT, DataCite, Dublin Core, or Schema.org naming conventions. The Rosetta-Stone Matrix simply makes these equivalences explicit so repositories can generate DRMP-conformant metadata with translation, not reinvention.

The matrix supports five core implementation tasks:

- Mapping: identify which existing metadata fields already satisfy DRMP requirements.
- Gap detection: determine which elements must be added to achieve Level 2 or Level 3.
- Normalization: standardize field names, data formats, and PID expressions across repositories.
- Federation readiness: ensure compatibility with OSTI, Data.gov, RE3Data, and integrated ecosystems like IRI or AmSC.

- Progress tracking: use RDA Gap ratings to estimate effort and maturity progress.

The full Rosetta-Stone Matrix is presented in Appendix B.

Together, these mappings show that the DRMP is not a new metadata standard but a harmonization layer. Repositories can generate DRMP-conformant metadata without restructuring their internal systems by simply aligning existing fields to the DRMP element set and progressively adding the few missing attributes required for Level 2 and Level 3 AI readiness.

This crosswalk provides the operational foundation for the implementation steps that follow in Sections 6.2–6.5.

6.2 Implementing the DRMP Using Existing Metadata Schemas

Most DOE repositories already maintain structured metadata that aligns with at least one of the six community schemas used in the DRMP crosswalk. The implementation process is therefore *transformational rather than generative* where repositories map existing fields into the DRMP element set and add only the few missing fields required at higher conformance levels.

6.2.1 Identify Your Source Schemas

Determine which community schemas your repository currently uses, directly or indirectly:

- Many institutional repositories expose **Dublin Core** fields.
- DOE-funded repositories often use **DCAT-US** through CKAN or custom REST APIs.
- DOI-issuing systems automatically generate **DataCite 4.6** metadata.
- Preservation-oriented repositories maintain **PREMIS** event records.

Knowing which schemas are already in use makes the DRMP mapping almost entirely mechanical.

6.2.2 Map Internal Fields to DRMP Elements

Using the Rosetta-Stone Matrix:

- repositoryName ← dct:title or dc:title or RE3Data repositoryName
- repositoryIdentifier ← DOI, ARK, HANDLE, or OSTI ID
- apiEndpoints ← DCAT endpointURL
- ontologyAlignment ← Schema.org about[] or DCAT theme taxonomy entries
- provenance ← PREMIS event records

This mapping is one-to-one and does not require repositories to modify internal storage structures.

6.2.3 Add Fields Needed for Level 2 or Level 3 Progression

Some repositories may not currently expose:

- license or retention policies
- access or authorization declarations
- harvest policies
- ontology integrations
- fixity or provenance summaries
- MCP tool descriptors (Level 3)

These can be added incrementally and published alongside existing metadata without system redesign.

By leveraging existing schemas and augmenting only where needed, repositories can achieve DRMP conformance with minimal overhead.

6.3 Building a DRMP Record from Local Metadata

Once mappings have been identified, repositories can construct a DRMP-compliant record using the JSON Schema in Appendix A. The process involves three steps:

Step 1: Gather Internal Metadata Fields
Pull values from internal databases, API outputs, repository configuration files, or published system documentation.

Step 2: Translate and Normalize
Apply the Rosetta-Stone Matrix to transform each field into the corresponding DRMP name and structured value. Normalization includes:

- converting dates to ISO 8601
- converting identifiers to resolvable PID forms
- listing metadata formats as strings
- using controlled vocabularies (e.g., ROR IDs, SPDX licenses) where applicable

Step 3: Generate the Record Automatically
Most repositories can implement a **lightweight export script** (Python, JavaScript, Go, or CLI) that:

- Pulls required metadata from internal systems
- Applies the mapping in a config file
- Outputs a JSON file that validates against the DRMP schema

This script can run as part of a scheduled build, deployment cycle, or metadata refresh pipeline. The outcome is a stable, machine-actionable representation that can be harvested by data and other orchestration services.

6.4 Validating DRMP Records

Validation is essential to ensure consistency across repositories and enable automated assessment, federated cataloging, and AI agent interaction.

6.4.1 Schema Validation

Appendix A provides a complete JSON Schema that can be validated using standard tools:

- Python: jsonschema
- Node.js: ajv
- Java: Everit or Jakarta JSON Processing
- CLI: jq + schema checkers

Validation ensures:

- required fields are present
- values conform to controlled vocabularies
- Level 2 and Level 3 records include all inherited fields from lower levels
- MCP metadata is correctly structured

6.4.2 Automated CI/CD Validation

Repositories are encouraged to incorporate:

- GitHub Actions
- GitLab CI
- Jenkins pipelines
- Argo Workflows

These pipelines can run a validation step whenever metadata changes, ensuring continuous compliance and producing machine-verifiable conformance status.

6.4.3 MCP Validation (Level 3 Only)

For AI-ready repositories, MCP manifests should be:

- hosted under /.well-known/mcp/
- validated using MCP-specific schema tools
- checked for completeness of tool descriptors and agent-facing capabilities

This enables seamless integration into AI reasoning systems and orchestration pipelines.

6.5 Security and Access Metadata

Security and access metadata must clearly describe how users and automated agents may interact with a repository without exposing *credentials, implementation details, or sensitive operational information*. DRMP metadata captures these controls through structured fields that provide high-level transparency while maintaining DOE cybersecurity compliance.

Access and Authentication Models
Repositories must populate the following fields to describe how access is governed:

- `accessModel` - Declares whether interfaces are open, authenticated, or restricted.
- `authorizationModel` - Specifies supported mechanisms (e.g., none, api-key, oauth2, federated identity).
- `accessMechanisms`[] - Documents available user and machine interfaces (REST API, OAI-PMH, WebUI, etc.).
- `authnMethods`[] - Lists recognized authentication workflows (OAuth2, DOE SSO, SAML, etc.).

These fields allow downstream systems such as including harvesters, workflows, and AI agents to correctly negotiate access requirements and handle restricted resources.

AI-Use and Harvest Policies
Two additional governance fields communicate usage boundaries:

- `aiUsePolicy` - Machine-readable constraints describing whether repository metadata and content may be used for training, derivative models, commercial purposes, or other AI-related activities.
- `harvestPolicy` - Declares machine-actionable rules for automated metadata harvesting and redistribution.

These fields prevent unintentional policy violations and enable automated systems to determine whether a repository can be used in training, inference, or federated search. Sample policies are provided in Appendix F.

Restricted Endpoints
For endpoints requiring authentication or role-based access, repositories should include a `conformsTo[]` reference pointing to publicly documented security profiles, access policies, or usage terms. This informs consuming systems about constraints without revealing sensitive internal configuration.

Handling Secrets
DRMP records must never contain API keys, passwords, tokens, or any operational credentials. Such information must be managed through standard cybersecurity practices outside of the metadata layer.

Thus, by representing access controls, authorization pathways, and AI-use rules in structured, machine-readable form, repositories provide transparency to users and automated agents while preserving security boundaries and supporting DOE-wide policy alignment.

6.6 Implementation Checklist for Repository Teams

This checklist distills the implementation process into actionable steps aligned with earlier chapters and appendices.

Step	Action	Reference
1. Determine Target Conformance Level	Select whether the repository will declare Level 1 (Minimum Viable Record), Level 2 (Interoperable), or Level 3 (AI-Ready).	Chapter 3 (Conformance Model)
2. Map Existing Metadata to DRMP Fields	Use the Crosswalk Matrix to map local metadata fields (RE3Data, DCAT, Dublin Core, DataCite, custom schema fields) to DRMP equivalents.	Chapter 6.1 + Appendix B (Rosetta Matrix)
3. Generate a DRMP Metadata Record	Construct a repository-level record conforming to the DRMP JSON Schema (Level-appropriate required fields).	Appendix A (JSON Schema + Examples)
4. Publish the DRMP Record in Machine-Actionable Form	Expose the DRMP record at a stable public endpoint (e.g., /drmp/v1/repository.json) in JSON-LD (preferred) or JSON.	Appendix A (Serialization Guidance)
5. Implement Required APIs (Level 2 and Level 3)	Expose machine-harvestable endpoints (REST or OAI-PMH), declare supported metadata formats, and provide license/retention/harvest policies.	Chapter 4.1 + Appendix C (API Guidance)
6. Publish Access & Authorization Metadata	Provide machine-readable accessModel, authorizationModel, and harvestPolicy to support automated harvesting and access by AI agents.	Chapter 6.4 (Access & Security)
7. Validate Metadata Using DRMP Tools	Run JSON Schema validation, API endpoint tests, and (for Level 3) semantic and policy checks. Integrate validation into CI/CD as appropriate.	Chapter 6.3 + Appendix A (Validation Tools)
8. Register or Integrate with External Catalogs	Register (or update) repository records with RE3Data, OSTI, DOE Data Explorer, or Data.gov using DRMP-aligned metadata.	Chapters 5 & 6.2 Mapping Guidance

Step	Action	Reference
9. Publish Transparency & Documentation	Provide a public landing page or repository README containing: conformance level, schema ID, change-log, contact information, and endpoint URLs.	Chapter 7 (Governance Guidance)
10. Implement MCP Context for AI-Ready Repositories (L3)	Expose MCP descriptors in /.well-known/mcp/ including aiUsePolicy, capabilities, and mcpTools declarations.	Chapter 4.2 + Appendix D (MCP Manifest)
11. Maintain Versioning & Sustainability	Version DRMP records, semantic context files, and APIs; follow deprecation guidance; maintain provenance and fixity for metadata changes.	Chapter 6.4 + Chapter 7 (Governance Chapter)

6.7 Summary

Chapter 6 operationalizes the DRMP by demonstrating how repositories can implement the profile using existing metadata, established community schemas, and lightweight validation workflows. Rather than requiring major system changes, the DRMP provides a practical alignment layer; it's a way to connect heterogeneous repository ecosystems across DOE while supporting the transparency, interoperability, and AI-readiness mandates emerging under federal policy and the AmSC.

Together with the conformance-level definitions (Chapter 3), the integration mechanisms (Chapter 4), and the schema-alignment rationale (Chapter 5), this implementation framework enables repositories to adopt the DRMP incrementally and sustainably while maintaining compatibility with DOE's evolving open-science and AI ecosystem.

7. Governance and Change Management

Chapters 3–6 described how repositories interpret, implement, and validate DRMP conformance. This chapter turns to the governance of the DRMP itself and how the profile will be maintained, updated, and stewarded as DOE research practices evolve.

The DCWG is a volunteer community of practice without formal oversight authority; its governance approach emphasizes transparency, collaborative review, and incremental improvement rather than prescriptive or mandatory requirements. The objective is to provide a stable, referenceable interoperability framework while allowing repositories full flexibility in local implementation.

7.1 Stewardship Model

- The DCWG serves as the steward of the DRMP, maintaining documentation, evaluating proposed changes, and coordinating input from across DOE laboratories, OSTI, and other stakeholder groups.
- Repositories and practitioners are encouraged to submit feedback, identify gaps, and propose extensions through the mechanisms described below.
- Significant revisions will be informed by broad consultation across DOE laboratories, program offices, OSTI, librarians, curators, technical professionals, and information science communities.
- Public copies of the specification, JSON schemas, examples, and crosswalk materials will be hosted on the DCWG website and GitHub repository.
- Major releases or formal versions may also be archived to OSTI to ensure persistent access, citation, and historical traceability.

7.2 Versioning

The DRMP follows *semantic versioning* to indicate the scope of change:

- **Patch (1.0.x):** editorial corrections, clarifications, or typographical fixes.
- **Minor (1.x.0):** additions of optional fields, new crosswalk alignments, or expanded best-practice guidance.
- **Major (x.0.0):** structural changes to conformance levels, revised definitions, or element deprecations.

Each release will include:

- A public changelog summarizing modifications and their rationale
- Notes on interoperability implications.
- Updated schemas, validation tools, and examples.

Version control ensures that repositories, tool developers, and federated services can track evolution of the DRMP and maintain alignment with specific profile version.

7.3 Deprecation Policy

- Elements identified for removal or major modification will be marked **deprecated** for at least one full release cycle before removal.

- Repositories should maintain backward compatibility during this period while preparing to adopt updated mappings.
- Deprecation warnings will appear in the DRMP documentation, crosswalk tables, JSON schemas, and example metadata records.
- Machine-actionable artifacts (e.g., JSON-Schema validators or metadata checklists) will include explicit annotations to assist implementers in migration and compliance.

7.4 Extensions and Community Contributions

The DRMP supports controlled, community-driven extensibility:

- Repositories may define local extensions using "other" enumerations or x--prefixed custom fields, provided semantic meaning is documented.
- Extensions demonstrating broad utility or alignment with DOE-wide priorities may be considered for inclusion in future releases.
- Contribution proposals should include clear use cases, example metadata, and evidence of semantic alignment with established ontologies, controlled vocabularies, or crosswalk targets.
- Extensions with substantial adoption across sites may be promoted to optional or recommended DRMP elements.

7.5 Change Request Process

The DCWG maintains a public issue tracker (e.g., GitHub) to coordinate updates:

- Submitters may propose clarifications, corrections, new fields, vocabulary updates, or crosswalk enhancements.
- Issues are triaged on a regular cadence and incorporated into the next appropriate release (patch, minor, major).
- Approved changes are propagated to all relevant artifacts, including the master mapping document, JSON schemas, examples, and validation tools, ensuring alignment between human-readable and machine-actionable representations.

7.6 Alignment with External Standards

To maintain interoperability, the DRMP is periodically reviewed against updates to major community and federal frameworks, including:

- RE3Data, DCAT-US v3, Schema.org, DataCite 4.6, Dublin Core (DCMI)
- FAIRsharing, OSTI metadata schemas, and emerging federal interoperability profiles
- Ontology services, provenance standards, and preservation frameworks relevant to AI-readiness

When upstream standards evolve, the DCWG evaluates impact on DOE practice and updates crosswalks accordingly. Major alignment changes will be documented with example adaptation strategies.

7.7 Long-Term Sustainability

- The DRMP represents a foundational investment in DOE's metadata infrastructure, supporting FAIR principles, interoperability, and AI-readiness across research programs.
- Continuous improvement, maintained through transparent governance, public documentation, and open participation, is essential for sustaining long-term relevance.
- During Calendar Year 2026, the DCWG will apply the DRMP to a set of pilot repositories to evaluate practicality, cost, and implementation burden, with findings incorporated into future releases.
- Beyond those early demonstrations, sustainability will rely on continued volunteer participation and integration with broader DOE initiatives such as the AmSC and Integrated Research Infrastructure (IRI).
- This ensures that the DRMP remains aligned with DOE's evolving strategic vision for interoperable, machine-actionable, and AI-ready scientific data ecosystems.

8. Conclusion

The DCWG developed this profile to provide practical, community-driven guidance for describing repositories across the DOE. By focusing on repository-level metadata (rather than dataset-level descriptions), the DRMP addresses a long-standing structural gap in discovery, interoperability, and machine-actionability within DOE's scientific data ecosystem.

The guidance presented here is advisory, not prescriptive. It is intended to support voluntary, incremental adoption across diverse repositories, enabling implementers to align with DOE's strategic direction while maintaining full flexibility in local system design.

We grounded the profile in widely used community schemas: RE3Data; DCAT-US v3; Schema.org; Dublin Core (DCMI 1.1); DataCite 4.6; and PREMIS 3.0. Thus, the DRMP avoids reinvention and strengthens interoperability with global data-exchange and FAIR-enabling infrastructures. These alignments ensure that DOE repositories can participate meaningfully in interconnected, multi-agency scientific knowledge environments.

We introduced three conformance levels in this report: Minimum Viable Record; Interoperable; and AI-Ready. This established a clear vocabulary and progression pathway for repositories to self-assess and enhance their metadata practices. Together with the JSON Schema, Rosetta-Matrix crosswalk, OpenAPI specification, and MCP templates provided in the appendices, the DRMP offers a coherent, implementation-ready framework that repositories can adopt at their own pace.

Data repositories operate with distinct missions, data types, stakeholder communities, and operational constraints. This diversity has naturally produced a patchwork of metadata practices and standards which, while locally optimized, often hinder broader discovery, access, and interoperability. Adoption of the DRMP can help introduce greater consistency across repositories; however, implementation also requires repositories to conduct self-assessments, identify conformance gaps, and establish improvement plans. These activities demand time, expertise, and resources, and in the absence of formal mandates may be perceived as optional, potentially slowing adoption. In practice, repositories frequently require clear justification for funding allocated to metadata enhancement efforts. Endorsement of the DRMP by DOE-level programs and initiatives, including the Genesis Mission and the AmSC, can provide important strategic validation and funding justification, analogous to policy-level drivers that have historically accelerated alignment efforts across the DOE complex.

Widespread, voluntary adoption of the DRMP will position DOE repositories to meet the goals of the One Big Beautiful Bill Act (2025), integrate into the AmSC, and maintain alignment with DOE's Public Access Plan 2.0. More importantly, it will strengthen the Department's long-term stewardship of scientific data by ensuring that repository metadata remains transparent, interoperable, durable, and AI-ready for emerging research paradigms.

The DCWG invites repositories across the DOE complex to engage with this guidance, propose enhancements, and contribute extensions. Through sustained collaboration and iterative improvement, the DOE community can collectively advance a resilient metadata framework, one

that supports discovery, accelerates access, and enables responsible innovation across the Department's scientific mission.

Appendices

The following appendices provide the machine-actionable resources, reference mappings, and validation templates that support implementation of the DOE Repository Metadata Profile (DRMP). They are intended for repository developers, data stewards, and information-management professionals who wish to align their local metadata exports with the DRMP conformance levels defined in Chapters 5 and 7.

Each appendix contains self-contained artifacts designed for reuse and integration within repository workflows:

- **Appendix A — JSON Schema:**
  Defines the core DRMP field structure and element types, including annotations for required, recommended, and optional attributes corresponding to Levels 1–3.
- **Appendix B — Crosswalk Matrix:**
  Presents the detailed field-by-field mappings between DRMP elements and community schemas such as RE3Data, DCAT-US v3, Schema.org, Dublin Core (DCMI 1.1), DataCite 4.6, and PREMIS 3.0; this serves as the canonical reference for semantic equivalence.
- **Appendix C — OpenAPI and XML Specifications:**
  Provides implementation examples for programmatic exposure of repository metadata through RESTful APIs and structured XML exports.
- **Appendix D — Model Context Protocol (MCP) Server Templates for AI Readiness:**
  Includes JSON-Schema validators, metadata-checklist templates, and example Model Context Protocol (MCP) messages to support automated validation and AI-assisted interoperability.
- **Appendix E –** DRMP Fields List:
  Provides a quick list of DRMP fields and a quick description based on conformance level.

These materials ensure that the DRMP remains both human-readable and machine-actionable, enabling consistent interpretation, implementation, and validation across the DOE repository landscape.

Appendix A - DOE Repository Metadata Profile Schema

Purpose and Context
This appendix defines the DOE Repository Metadata Profile (JSON Schema)—a machine-actionable specification that validates conformance across the three maturity levels of this profile. It supports automated metadata assessment within DOE repository workflows, enabling continuous integration (CI) pipelines, metadata quality dashboards, and reproducibility audits.

The schema implements:

- Conditional validation based on the repository's declared "conformsTo" level (Level-1-MVR, Level-2-Interoperable, Level-3-AIReady);
- Explicit PREMIS 3.0 objects for provenance and fixity;
- Embedded RDA anchor annotations for traceability; and
- Extensibility through $defs for future schema versioning.

Repositories are encouraged to publish their metadata in JSON-LD and validate it against this schema before registration or harvest.

Validation ensures consistency across DOE repositories and interoperability with external catalogs such as RE3Data, DataCite, and DCAT-US v3.

Schema Overview

- File name: doe-repository-metadata-profile-v1.0.json
- Schema version: 1.0 (Draft 2025-11)
- Compliance levels:
  - Level 1 — Minimum Viable Record
  - Level 2 — Interoperable
  - Level 3 — AI-Ready

DOE Repository Metadata Profile (DRMP)

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://example.org/schemas/drmp.json",
  "title": "DOE Repository Metadata Profile (DRMP)",
  "description": "Repository-level metadata profile for DOE repositories,
with conformance levels for Minimum Viable Record (Level 1), Interoperable
(Level 2), and AI-Ready (Level 3).",
  "type": "object",

  "properties": {
    "repositoryIdentifier": { "type": "string" },
    "repositoryName": { "type": "string" },
    "repositoryType": { "type": "string" },
    "hostingInstitution": { "type": "string" },
    "contactPoint": { "type": "string" },

    "repositoryDescription": {
      "type": "string",
```

```
      "description": "Human-readable description of the repository's scope,
mission, content focus, and community served."
    },

    "themeTaxonomy": {
      "type": "array",
      "items": { "type": "string" },
      "description": "Repository-level subject schemes, taxonomies, or
FAIRsharing identifiers."
    },

    "fundingBody": {
      "type": "array",
      "items": { "type": "string" },
      "description": "Organizations or agencies that fund or sponsor the
repository."
    },

    "conformsTo": {
      "type": "string",
      "enum": ["Level-1-MVR", "Level-2-Interoperable", "Level-3-AIReady"]
    },

    "apiEndpoints": {
      "type": "array",
      "items": { "type": "string", "format": "uri" }
    },

    "metadataFormats": {
      "type": "array",
      "items": { "type": "string" },
      "description": "Repository-level metadata formats or schemas exposed by
the repository (not dataset-level formats)."
    },

    "licensePolicy": { "type": "string", "format": "uri" },
    "retentionPolicy": { "type": "string", "format": "uri" },
    "harvestPolicy": { "type": "string", "format": "uri" },

    "issued": { "type": "string", "format": "date" },
    "modified": { "type": "string", "format": "date" },
    "stewardshipCommitment": { "type": "string" },

    "persistentIdentifierPolicy": {
      "type": "object",
      "properties": {
        "pidSystems": {
          "type": "array",
          "items": {
            "type": "string",
            "enum": ["DOI", "ARK", "Handle", "ORCID", "ROR", "LocalID",
"None", "Other"]
          }
        },
        "pidPolicy": { "type": "string", "format": "uri" }
      },
      "required": ["pidSystems"]
```

```
    },

    "accessModel": { "type": "string" },
    "authorizationModel": { "type": "string" },

    "capabilities": {
      "type": "array",
      "items": {
        "type": "string",
        "enum": [
          "api-access", "bulk-download", "schema-browse",
          "validation-service", "webhooks", "dataset-versioning",
          "search-api", "metadata-export", "other"
        ]
      }
    },

    "ontologyAlignment": {
      "type": "array",
      "items": { "type": "string", "format": "uri" }
    },

    "ontologyVersion": {
      "type": "array",
      "items": { "type": "string", "format": "uri" }
    },

    "preservationPolicy": { "type": "string", "format": "uri" },

    "provenance": { "type": "object", "additionalProperties": true },
    "fixity": { "type": "object", "additionalProperties": true },

    "aiUsePolicy": {
      "type": "object",
      "properties": {
        "trainingPermission": {
          "type": "string",
          "enum": ["allowed", "prohibited", "restricted", "metadata-only",
"dataset-specific", "unspecified"]
        },
        "derivativeModelPermission": {
          "type": "string",
          "enum": ["allowed", "prohibited", "restricted", "dataset-specific",
"unspecified"]
        },
        "commercialAIUse": {
          "type": "string",
          "enum": ["allowed", "noncommercial-only", "prohibited", "dataset-
specific", "unsspecified"]
        },
        "modelEvaluationPermission": {
          "type": "string",
          "enum": ["allowed", "prohibited", "restricted", "dataset-specific",
"unspecified"]
        },
        "agentInteractionAllowed": {
          "type": "string",
```

```
          "enum": ["allowed", "prohibited", "restricted", "dataset-specific",
"unspecified"]
        },
        "aiPolicy": { "type": "string", "format": "uri" },
        "notes": { "type": "string" }
      },
      "required": ["trainingPermission"]
    },

    "mcpTools": {
      "type": "array",
      "items": { "type": "string" }
    },

    "semanticContext": {
      "type": "array",
      "items": { "type": "string", "format": "uri" }
    }
  },

  "required": [
    "repositoryIdentifier",
    "repositoryName",
    "repositoryType",
    "hostingInstitution",
    "contactPoint",
    "conformsTo"
  ],

  "allOf": [

    {
      "if": {
        "properties": { "conformsTo": { "const": "Level-1-MVR" } }
      },
      "then": {
        "required": [
          "repositoryIdentifier",
          "repositoryName",
          "repositoryType",
          "hostingInstitution",
          "contactPoint",
          "conformsTo"
        ]
      }
    },

    {
      "if": {
        "properties": { "conformsTo": { "const": "Level-2-Interoperable" } }
      },
      "then": {
        "required": [
          "apiEndpoints",
          "metadataFormats",
          "licensePolicy",
          "retentionPolicy",
```

```
          "harvestPolicy",
          "issued",
          "modified",
          "stewardshipCommitment",
          "persistentIdentifierPolicy"
        ]
      }
    },

    {
      "if": {
        "properties": { "conformsTo": { "const": "Level-3-AIReady" } }
      },
      "then": {
        "required": [
          "accessModel",
          "authorizationModel",
          "capabilities",
          "ontologyAlignment",
          "preservationPolicy",
          "provenance",
          "fixity",
          "aiUsePolicy",
          "mcpTools"
        ]
      }
    }
  ]
}
```

Example: Level 1 Compliant Metadata Record

A generic project-level repository for a small DOE-funded research initiative.

Use case: A small project hosting workflow outputs on a project Git server. Needs only L1 metadata to integrate with automation.

```
{
  "repositoryIdentifier": "proj-thermo-2025",
  "repositoryName": "ThermoSim Workflow Outputs",
  "repositoryType": "project-repository",
  "hostingInstitution": "Hydrogen Materials Simulation Project",
  "contactPoint": "thermosim-support@example.org",
  "conformsTo": "Level-1-MVR",

  "repositoryDescription": "Lightweight repository hosting workflow-generated
datasets and simulation artifacts for a multi-institution DOE project on
hydrogen storage materials."
}
```

Example: Level 2 Compliant Metadata Record

A fake example for the National Energy Technology Laboratory (NETL).

Use case: NETL wants to expose structured metadata, machine harvest endpoints, and PID policy for an operational repository.

```
{
  "repositoryIdentifier": "netl-datahub",
  "repositoryName": "NETL DataHub",
  "repositoryType": "federal-research-repository",
  "hostingInstitution": "National Energy Technology Laboratory",
  "contactPoint": "datahub@netl.doe.gov",
  "conformsTo": "Level-2-Interoperable",

  "apiEndpoints": [
    "https://datahub.netl.doe.gov/api/v1/",
    "https://datahub.netl.doe.gov/oai"
  ],
  "metadataFormats": ["DCAT-US-3", "DataCite-4.6", "EML"],
  "licensePolicy": "https://netl.doe.gov/datahub/policies/license",
  "retentionPolicy": "https://netl.doe.gov/datahub/policies/retention",
  "harvestPolicy": "https://netl.doe.gov/datahub/policies/harvest",

  "issued": "2019-04-21",
  "modified": "2025-02-01",
  "stewardshipCommitment": "NETL maintains public datasets for long-term
reuse through federal data management requirements.",

  "persistentIdentifierPolicy": {
    "pidSystems": ["DOI", "ORCID", "ROR"],
    "pidPolicy": "https://netl.doe.gov/datahub/policies/persistent-
identifiers"
  },

  "repositoryDescription": "The NETL DataHub provides access to experimental,
simulation, and field datasets supporting fossil energy, carbon management,
and grid reliability research.",

  "themeTaxonomy": ["Energy Systems", "Carbon Capture", "Combustion"],
  "fundingBody": ["DOE Office of Fossil Energy and Carbon Management"]
}
```

Example: Level 3 Compliant Metadata Record

A fake example for a fully AI-ready institutional repository linked to automation and semantic alignment for OLCF.

Use case: OLCF exposes MCP tools, semantic contexts, fixity, provenance, and full AI policy for machine agents.

```
{
  "repositoryIdentifier": "olcf-datascape",
```

```
  "repositoryName": "OLCF DataScape",
  "repositoryType": "institutional-repository",
  "hostingInstitution": "Oak Ridge Leadership Computing Facility (ORNL)",
  "contactPoint": "datascape-support@ornl.gov",
  "conformsTo": "Level-3-AIReady",

  "apiEndpoints": [
    "https://datascape.olcf.ornl.gov/api/v2/",
    "https://datascape.olcf.ornl.gov/search-api/"
  ],

  "metadataFormats": ["DCAT-US-3", "schema.org/Dataset", "DataCite-4.6"],
  "licensePolicy": "https://datascape.olcf.ornl.gov/policies/license",
  "retentionPolicy": "https://datascape.olcf.ornl.gov/policies/retention",
  "harvestPolicy": "https://datascape.olcf.ornl.gov/policies/harvest",

  "issued": "2017-09-01",
  "modified": "2025-02-02",

  "stewardshipCommitment": "OLCF provides long-term stewardship for datasets
produced on DOE supercomputers to ensure reproducibility, transparency, and
reuse.",

  "persistentIdentifierPolicy": {
    "pidSystems": ["DOI", "ORCID", "Handle", "ROR"],
    "pidPolicy": "https://datascape.olcf.ornl.gov/policies/persistent-
identifiers"
  },

  "accessModel": "open-access-with-registration",
  "authorizationModel": "federated-authentication-ORCID-ROR-DOE-PIV",

  "capabilities": [
    "api-access",
    "bulk-download",
    "schema-browse",
    "validation-service",
    "search-api",
    "metadata-export"
  ],

  "ontologyAlignment": [
    "https://schema.org/",
    "https://w3id.org/fair/ontology",
    "https://linked.data.gov/def/dcat"
  ],

  "ontologyVersion": [
    "https://schema.org/version/24.0",
    "https://datacite.org/schema/kernel-4.6"
  ],

  "preservationPolicy":
"https://datascape.olcf.ornl.gov/policies/preservation",

  "provenance": {
    "pipeline": "GitHub Actions + OLCF Workflows",
```

```
    "commit": "f29ce12a",
    "validatedOn": "2025-01-30"
  },

  "fixity": {
    "sha256": "c3299bd3d9470f9767ea12487f7ab02a4c8d36fc2a6f3522fcb8ac",
    "verified": "2025-01-30"
  },

  "aiUsePolicy": {
    "trainingPermission": "metadata-only",
    "derivativeModelPermission": "restricted",
    "commercialAIUse": "noncommercial-only",
    "modelEvaluationPermission": "allowed",
    "agentInteractionAllowed": "restricted",
    "aiPolicy": "https://datascape.olcf.ornl.gov/policies/ai-governance",
    "notes": "Model training permitted only on curated metadata; dataset
training requires explicit request."
  },

  "mcpTools": [
    "olcf-indexer",
    "semantic-linker",
    "data-quality-agent",
    "schema-validator"
  ],

  "semanticContext": [
    "https://datascape.olcf.ornl.gov/context/core.jsonld",
    "https://datascape.olcf.ornl.gov/context/ai-governance.jsonld"
  ],

  "repositoryDescription": "OLCF DataScape is the institutional repository
supporting AI-ready workflows, provenance-enabled research, and MCP-based
automation for datasets generated on DOE supercomputing systems.",

  "themeTaxonomy": ["HPC", "Fusion", "Materials", "Climate"],
  "fundingBody": ["DOE Office of Science", "ASCR"]
}
```


Implementation Notes

A. Validation Tools

DRMP conformance can be validated using any JSON Schema validator that fully supports draft-2020-12, including jsonschema (Python), ajv (Node.js), and spectral.

Repositories should validate both:

- baseline schema conformance, and
- conformance-level–specific requirements (L1, L2, L3).

Level 3 repositories should additionally validate MCP tool descriptors and ensure all MCP endpoints return well-formed responses.

B. Continuous Integration and Deployment
Integrating DRMP validation into CI/CD pipelines ensures that:

- repository metadata updates do not break conformance,
- API endpoints remain reachable and machine-actionable,
- MCP tool descriptors stay synchronized with repository capabilities.

Recommended CI checks include:

1. JSON Schema validation (all levels)
2. API health checks (Level 2 and above)
3. MCP tool descriptor validation (Level 3)
4. Fixity and provenance updates (where applicable)

C. Schema Versioning and Compatibility
Future revisions of the DRMP should adopt clear semantic versioning:

- MAJOR: Breaking changes to fields, required elements, or conformance-level meaning
- MINOR: Additive extensions, optional fields, new capabilities
- PATCH: Error fixes, clarifications

Each schema must publish a unique $id and include a schemaVersion field in its metadata to ensure AI agents and MCP tooling can negotiate compatibility across versions.

Version stability is particularly important for Level 3 repositories that expose MCP interfaces.

D. Governance, Stewardship, and Change Control
The DRMP schema and associated conformance model should be governed by the DCWG with transparent community change control and public review.

A central, publicly accessible schema registry should be maintained to publish:

- stable schema versions,
- deprecation notices,
- MCP tool updates,
- implementation patterns,
- and crosswalk/mapping profile revisions.

Repositories should maintain internal governance processes for:

- metadata provenance,
- fixity tracking,
- AI use policy management, and
- stewardship commitments (especially for long-term preservation claims).

Appendix B — Crosswalk Matrix

Purpose and Scope
This appendix provides the **canonical crosswalk matrix** that aligns each metadata element defined in the DOE Repository Metadata Profile (DRMP) with equivalent or semantically related terms from six community schemas: **RE3Data 4.0, DCAT-US v3, Schema.org, Dublin Core (DCMI 1.1), DataCite 4.6, and PREMIS 3.0**.

These mappings constitute the operational foundation of the DRMP, enabling consistent translation of repository-level metadata across diverse discovery platforms and interoperability frameworks.

The crosswalk serves three complementary purposes:

1. **Semantic Equivalence:** establish a transparent alignment between DRMP metadata elements and existing global standards, allowing metadata to be programmatically transformed without loss of meaning.
2. **Conformance Traceability:** map required and recommended fields to the DRMP conformance levels defined in Chapter 3 (*Minimum Viable Record, Interoperable,* and *AI-Ready*) supporting automated validation and self-assessment.
3. **RDA Attributes Awareness:** integrate RDA Repository Attributes (1–17) from the *RDA Common Descriptive Attributes of Research Data Repositories (2024, DOI: 10.15497/rda00103)* to indicate typical implementation effort and maturity for each field.

Each table row represents a single DRMP element and lists:

- Corresponding predicates or property names in the six community schemas;
- The applicable DRMP conformance level(s); and
- The associated RDA anchor number.

Usage Guidance
Implementers can reference this matrix to:

- Align local repository metadata exports with DRMP conformance levels;
- Identify missing fields or deprecated elements during modernization efforts; and
- Facilitate bidirectional mapping between DOE repositories and federal open-data registries such as RE3Data, OSTI, and Data.gov.

DRMP Field	L1	L2	L3	RDA#	re3data 4.0	DCAT-US v3	Schema.org	Dublin Core	DataCite 4.6	PREMIS 3.0
accessModel	—	✓ R	✓	14	r3d:dataAccess; r3d:databaseAccess	dct:accessRights	conditionsOfAccess; isAccessibleForFree	dcterms:accessRights	rights	rights
aiUsePolicy	—	—	✓	15	r3d:policy	dct:rights	usageInfo	dcterms:rights	rights	rightsStatement
apiEndpoints	—	✓	✓	9	r3d:apiAccess	dcat:endpointURL	distribution	—	—	—
authorizationModel	—	✓ R	✓	14	r3d:access; r3d:dataAccessRestriction; r3d:databaseAccessRestriction	dct:accessRights	audience	dcterms:accessRights	rights; rightsURI	rights
capabilities	—	✓ R	✓	10	r3d:apiAccess with r3d:databaseAccess; r3d:api/r3d:databaseAccess	dcat:service	Service; Action	—	—	environment
contactPoint	✓	✓	✓	6	r3d:contact	dcat:contactPoint	contactPoint	dc:contributor	contributor	agentIdentifier
fixity	—	—	✓	17	—	spdx:checksum	checksum	—	checksum	objectCharacteristics.fixity
fundingBody	✓ R	✓	✓	7	r3d:fundingInformation	dct:fundedBy	funder	dc:contributor	fundingReference	—
harvestPolicy	—	—	✓	14	r3d:harvestType	dct:accrualMethod (closer to harvesting) and dct:accessRights	—	dc:relation	—	eventType
hostingInstitution	✓	✓	✓	5	r3d:institution	dct:publisher	provider	dc:publisher	publisher	agentName
issued	—	✓	✓	11	r3d:metadataUpdate	dct:issued	datePublished	dc:date	publicationYear	eventDateTime
licensePolicy	—	✓	✓	15	r3d:policy	dct:license	license	dcterms:license	rightsURI	rightsStatement
mcpTools	—	—	✓	10	—	dct:service	instrument	—	resourceType	environment
metadataFormats	—	✓	✓	10	r3d:metadataStandard	dct:conformsTo	encodingFormat	dc:format	metadataScheme	objectCharacteristics
modified	—	✓	✓	11	r3d:metadataUpdate	dct:modified	dateModified	dc:date	publicationYear	eventDateTime
ontologyAlignment	—	—	✓	13	—	dct:conformsTo	about	dc:subject	—	—
ontologyVersion	—	—	✓	13	—	dct:hasVersion	version	dc:hasVersion	—	—
persistentIdentifierPolicies	—	✓	✓	2	r3d:policy	dct:license; dct:policy	policy	dcterms:rights; dcterms:policy	rightsURI	rightsBasis
preservationPolicy	—	—	✓	17	r3d:preservationPolicy	dct:policy	conditionsOfAccess	dcterms:policy	rights	preservation
provenance	—	—	✓	12	—	dct:provenance	provenance	dcterms:provenance	—	event / eventOutcome
repositoryDescription	✓ R	✓ R	✓ R	7	r3d:description	dct:description	description	dcterms:description	description	—
repositoryIdentifier	✓	✓	✓	2	r3d:identifier	dct:identifier	identifier	dc:identifier	identifier	objectIdentifier
repositoryName	✓	✓	✓	3	r3d:repositoryName	dct:title	name	dc:title	title	—
repositoryType	✓	✓	✓	4	r3d:repositoryType	dct:type	additionalType	dc:type	resourceTypeGeneral	objectCategory
retentionPolicy	—	✓	✓	17	r3d:policy	dct:accessRights	—	dc:rights	—	preservationLevel
semanticContext	—	—	✓	13	—	—	@context	dcterms:relation	—	semanticUnit
stewardshipCommitment	—	✓	✓	12	—	dct:accrualPolicy	—	dc:relation	—	rightsBasis
themeTaxonomy	—	—	✓	13	—	dcat:theme	keywords	dc:subject	subjectScheme	—

Appendix C — OpenAPI and XML Specifications

C.1 Purpose and Scope

This appendix provides lightweight reference examples showing how core DRMP elements can be exposed through machine-readable APIs and XML-based interfaces. These examples assist repositories in achieving Level-2 interoperability by offering minimal, adaptable templates. The content here is not prescriptive; repositories may tailor these examples to their technical platforms, institutional requirements, and governance structures.

C.2 Minimal OpenAPI 3.1 Example

Below is a minimal OpenAPI 3.1 snippet illustrating how DRMP fields may be represented in a standardized API:

```
openapi: 3.1.0
info:
  title: DRMP Repository Metadata API
  version: "1.0.0"

paths:
  /drmp/v1/repository:
    get:
      summary: Retrieve repository-level DRMP metadata
      responses:
        "200":
          description: DRMP-compliant repository record
          content:
            application/json:
              schema:
                $ref: "#/components/schemas/RepositoryRecord"

components:
  schemas:
    RepositoryRecord:
      type: object
      required:
        - repositoryIdentifier
        - repositoryName
        - repositoryType
        - hostingInstitution
        - contactPoint
        - conformsTo
      properties:
        repositoryIdentifier:
          type: string
        repositoryName:
          type: string
        repositoryType:
          type: string
        hostingInstitution:
          type: string
```

```
        contactPoint:
          type: string
        conformsTo:
          type: string
          enum: ["Level-1-MVR", "Level-2-Interoperable", "Level-3-AIReady"]
        apiEndpoints:
          type: array
          items: { type: string, format: uri }
        metadataFormats:
          type: array
          items: { type: string }
        licensePolicy:
          type: string
          format: uri
```

C.3 Minimal XML/XSD Representation Example

C.3.1 XML Schema Definition (XSD)

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
           targetNamespace="https://example.org/drmp"
           xmlns="https://example.org/drmp"
           elementFormDefault="qualified">

  <xs:element name="repositoryRecord" type="RepositoryRecord"/>

  <xs:complexType name="RepositoryRecord">
    <xs:sequence>
      <xs:element name="repositoryIdentifier" type="xs:string"/>
      <xs:element name="repositoryName" type="xs:string"/>
      <xs:element name="repositoryType" type="xs:string"/>
      <xs:element name="hostingInstitution" type="xs:string"/>
      <xs:element name="contactPoint" type="xs:string"/>
      <xs:element name="conformsTo" type="xs:string"/>
      <xs:element name="apiEndpoints" type="xs:anyURI" minOccurs="0"
maxOccurs="unbounded"/>
      <xs:element name="metadataFormats" type="xs:string" minOccurs="0"
maxOccurs="unbounded"/>
      <xs:element name="licensePolicy" type="xs:anyURI" minOccurs="0"/>
    </xs:sequence>
  </xs:complexType>

</xs:schema>
```

C.3.2 Example XML Record

```
<repositoryRecord xmlns="https://example.org/drmp">
  <repositoryIdentifier>netl-datahub</repositoryIdentifier>
  <repositoryName>NETL DataHub</repositoryName>
  <repositoryType>federal-research-repository</repositoryType>
  <hostingInstitution>National Energy Technology Laboratory
```

```
      </hostingInstitution>
  <contactPoint>datahub@netl.doe.gov</contactPoint>
  <conformsTo>Level-2-Interoperable</conformsTo>
  <apiEndpoints>https://datahub.netl.doe.gov/api/v1/</apiEndpoints>
  <metadataFormats>DCAT-US-3</metadataFormats>

<licensePolicy>https://datahub.netl.doe.gov/policies/license</licensePolicy>
</repositoryRecord>
```

C.4 Implementation Guidance

Repositories are not required to expose DRMP metadata using OpenAPI or XML, but providing at least one machine-readable interface is strongly recommended for Level-2 interoperability. These examples support harvesting, catalog synchronization, MCP tool integration, and automated FAIR or AI-readiness assessments. Repositories may adapt or extend these templates per organizational requirements.

Appendix D — Model Context Protocol (MCP) Server Templates for AI Readiness

Purpose and Scope
This appendix defines the **Model Context Protocol (MCP)** templates that enable repositories to expose DRMP-conformant metadata as structured, machine-interpretable context for **AI systems**, including large-language-model agents, validation services, and metadata reasoning frameworks.

These templates operationalize the **AI-Ready (Level 3)** tier of the DOE Repository Metadata Profile by providing a lightweight, standards-based interface that connects repository metadata to the emerging **AI research ecosystem**.

The goal of these MCP templates is to:

- Allow repositories to advertise their **conformance level**, schema alignment, and API capabilities in a consistent, machine-discoverable way.
- Support AI applications in dynamically retrieving, validating, and contextualizing repository metadata.
- Provide a bridge between **static DRMP records** and **dynamic AI reasoning environments** that can leverage metadata for search, provenance analysis, and policy awareness.

D.1 Design Overview

Each MCP deployment consists of a minimal set of files that together describe a repository's context and capabilities:

File	Purpose
manifest.json	Declares repository identity, schema reference, DRMP conformance level, and available MCP tools.
context.json / context.jsonld	Publishes live metadata context aligned to the DRMP v1.0 schema.
tools/validate.json	Defines a validation tool for schema-level or JSON-Schema conformance checks.
tools/search.json	(Optional) Implements a query tool for searching repository metadata or records.

The structure follows the **OpenAI Model Context Protocol** specification, ensuring interoperability with common AI agents and orchestration frameworks.

D.2 Core Fields in MCP Manifest

Field	Description	Example
name	Canonical MCP endpoint name	"ornl-data-mcp"

description	Short summary of repository scope and purpose	"ORNL Open Data Repository MCP Interface"
version	MCP manifest version	"1.0"
conformsTo	Declared DRMP level(s)	["Level-2-Interoperable","Level-3-AIReady"]
schema	JSON Schema URL reference	"https://data.ornl.gov/schema/doe-repository-metadata-profile-v1.0.json"
api	Public OpenAPI specification or REST endpoint	"https://data.ornl.gov/openapi/repository.json"
endpoints	List of available MCP tools or actions	["validate","search","summarize"]
auth	Supported authentication mechanisms	["none","apiKey","OAuth2"]

D.3 Example MCP Manifests

(a) Minimal Level 2 — Interoperable Repository

```
{
  "name": "ornl-data-mcp",
  "description": "ORNL Open Data Repository MCP Interface for DOE
interoperability.",
  "version": "1.0",
  "conformsTo": ["Level-2-Interoperable"],
  "schema": "https://data.ornl.gov/schema/doe-repository-metadata-profile-
v1.0.json",
  "api": "https://data.ornl.gov/openapi/repository.json",
  "endpoints": ["validate", "search"],
  "auth": ["none"]
}
```

(b) Level 3 — AI-Ready Extension

```
{
  "name": "ornl-data-mcp-ai",
  "description": "AI-ready MCP server exposing contextual metadata for
reasoning agents.",
  "version": "1.0",
  "conformsTo": ["Level-3-AIReady"],
  "schema": "https://data.ornl.gov/schema/doe-repository-metadata-profile-
v1.0.json",
  "context": "https://data.ornl.gov/context/repository.jsonld",
  "api": "https://data.ornl.gov/openapi/repository.json",
  "endpoints": ["validate", "summarize", "semantic-search"],
  "auth": ["apiKey"]
}
```

D.4 Implementation Guidance

- **Publication Location:** MCP manifests should be hosted at a well-known path (e.g., /.well-known/mcp/manifest.json) to support automated discovery by AI tools.
- **Validation Integration:** The validate tool invokes the DOE Repository Metadata Profile schema (Appendix A) to check conformance and report gap levels.
- **Context Enrichment:** Level 3 repositories should publish a context.jsonld file that declares ontology links, PREMIS provenance relations, and semantic contexts for AI-based reasoning.
- **Security Practices:** No authentication secrets should be embedded in manifests. Repositories may reference public documentation of their OAuth2 or DOE SSO policies through authDocs links.
- **Discovery and Harvesting:** MCP endpoints enable AI agents to discover repositories, summarize metadata quality, and suggest alignment improvements based on DRMP gap scores.
- **Lifecycle Updates:** The manifest version and schema URL should be updated in tandem with official DRMP releases to preserve backward compatibility.

D.5 Relationship to the DRMP Framework

DRMP Level	MCP Integration Expectation	Outcome
Level 1 – MVR	Optional static manifest with basic repository contact info.	Discovery-only exposure.
Level 2 – Interoperable	Required manifest and validation tool for federated integration.	Enables federated indexing and schema harvesting.
Level 3 – AI-Ready	Full MCP implementation including contextual metadata and semantic interfaces.	Supports AI-assisted metadata reasoning and automated trust evaluation.

D.6 Summary

MCP server templates extend the DOE Repository Metadata Profile into the AI domain by making repository-level metadata machine-interpretable in real time. They provide a lightweight on-ramp for AI-assisted data discovery, metadata validation, and contextual integration across DOE and federated research environments.

Through voluntary adoption of these templates, repositories can ensure their metadata is not only FAIR but also **AI-ready**, advancing DOE's vision for interoperable, transparent, and intelligent scientific data stewardship.

Appendix D-1 — OpenAPI & MCP Endpoint Reference

Purpose
This appendix standardizes **endpoint names, request/response models, discovery paths, and example URLs** for DRMP-aware **OpenAPI** services and **MCP** servers. It enables consistent client behavior across repositories and supports automated discovery by AI agents.

This implementation conforms to the **Anthropic Model Context Protocol (MCP) v1.0** specification (2025), as published by Anthropic Inc., and adapts that framework for **DOE repository metadata interoperability and AI-readiness**.

Reference: Anthropic Inc. (2025). **Model Context Protocol Specification v1.0.** Available at: https://modelcontextprotocol.io

D-1.1 Discovery & Manifests

Artifact	Path (recommended)	Content	Example
MCP Manifest	/.well-known/mcp/manifest.json	Declares service identity, DRMP schema URL, conformance level(s), tools	https://repo.example.org/.well-known/mcp/manifest.json
MCP Context (Level 3)	/.well-known/mcp/context.jsonld	JSON-LD context linking DRMP fields, ontologies, PREMIS	https://repo.example.org/.well-known/mcp/context.jsonld
OpenAPI Spec	/openapi/repository.json	OpenAPI 3.1 descriptor mapping to DRMP fields	https://repo.example.org/openapi/repository.json
DRMP JSON Schema	/schema/doe-repository-metadata-profile-v1.0.json	Canonical schema URL	https://repo.example.org/schema/doe-repository-metadata-profile-v1.0.json

D-1.2 Standard OpenAPI Endpoints

Base URL: https://repo.example.org/api

Endpoint	Method	Purpose	Required Params	Response (HTTP 200)
/drmp/v1/repository	GET	Return repository-level DRMP record	format (optional: json, jsonld)	application/json (DRMP object)
/drmp/v1/validate	POST	Validate a DRMP record against schema	Body: DRMP JSON	{ "ok": bool, "level": "L1/L2/L3", "errors": [...] }
/drmp/v1/search	GET	Search repository metadata (simple)	q (string), limit, offset	{ "items": [...], "total": n }
/drmp/v1/harvest	GET	List harvestable endpoints	—	{ "oai": "...", "dcat": "...", "dumps": [...] }
/drmp/v1/capabilities	GET	Expose feature flags & auth modes	—	{ "conformsTo": [...], "auth": [...], "versions": {...} }

Example: /drmp/v1/repository (JSON)

```
{
  "repositoryIdentifier": "doi:10.5439/1767341",
  "repositoryName": "ORNL Open Data Repository",
  "repositoryType": "institutional",
  "hostingInstitution": "Oak Ridge National Laboratory",
  "contactPoint": "data-curation@ornl.gov",
  "apiEndpoints": ["https://repo.example.org/api/drmp/v1/repository"],
  "metadataFormats": ["Dublin Core", "DataCite 4.6"],
  "licensePolicy": "https://creativecommons.org/licenses/by/4.0/",
  "retentionPolicy": "https://repo.example.org/policy/retention",
  "issued": "2024-03-01T00:00:00Z",
  "modified": "2025-09-15T00:00:00Z",
  "ontologyAlignment": ["https://purl.obolibrary.org/obo/envo.owl"],
  "preservationPolicy": "https://repo.example.org/policy/preservation",
  "conformsTo": ["Level-2-Interoperable"]
}
```

Example: /drmp/v1/validate (JSON)

```
{
  "ok": true,
  "level": "L2",
  "schema": "https://repo.example.org/schema/doe-repository-metadata-profile-
v1.0.json",
  "errors": [],
  "deprecated": [],
```

```
  "rdaSummary": { "gap1": 7, "gap2": 5, "gap3": 0 }
}
```

D-1.3 MCP Tool Conventions

MCP servers expose **tools** (logical operations) that AI agents can call. Minimum recommended set:

Tool Name	Purpose	Input	Output
validate	Validate DRMP JSON record	{ "record": {…} }	{ "ok": bool, "level": "L1/L2/L3", "errors": [...] }
describe	Summarize repository & conformance	{}	{ "name": "...", "conformsTo": [...], "schema": "...", "api": "..." }
search	Keyword search	{ "q": "string", "limit": 20, "offset": 0 }	{ "items": [...], "total": n }
capabilities	List features & auth methods	{}	{ "auth": [...], "endpoints": [...], "versions": {...} }

MCP Tool: validate (example IO)

```
// input
{ "record": { "...": "DRMP fields here" } }

// output
{
  "ok": false,
  "level": "L1",
  "errors": [
    { "path": "/licensePolicy", "message": "required for Level-2-
Interoperable" }
  ],
  "hint": "Add licensePolicy to reach Level-2."
}
```

MCP Tool: describe (example output)

```
{
  "name": "ornl-data-mcp",
  "description": "ORNL Open Data Repository MCP Interface",
  "conformsTo": ["Level-2-Interoperable"],
  "schema": "https://repo.example.org/schema/doe-repository-metadata-profile-
v1.0.json",
  "api": "https://repo.example.org/openapi/repository.json",
  "auth": ["none"],
  "endpoints": ["validate","search","describe"]
}
```

D-1.4 Versioning & Namespaces

Item	Recommendation	Example

DRMP Schema $id	Versioned path per release	.../schema/doe-repository-metadata-profile-v1.0.json
OpenAPI info.version	Match DRMP schema minor	1.0.0
MCP version	Track manifest version	"1.0"
Namespace in JSON-LD	Stable, versioned	https://repo.example.org/ns/drmp/v1#

D-1.5 Authentication & Security Notes

- No secrets in manifests or DRMP records.
- Advertise supported modes via auth: ["none","apiKey","OAuth2"].
- If access-controlled, include a public authDocs URL (e.g., OAuth2 flows, DOE SSO).
- For OpenAPI, use securitySchemes and per-operation security blocks.

D-1.6 Error & Health Conventions

Endpoint	Health Check	Error Payload (JSON)
Any	GET /health → {"status":"ok","version":"1.0.0"}	{ "error": "BadRequest", "message": "...", "traceId": "uuid" }

D-1.7 Example URL Patterns (Canonical)

```
https://repo.example.org/.well-known/mcp/manifest.json
https://repo.example.org/.well-known/mcp/context.jsonld
https://repo.example.org/openapi/repository.json
https://repo.example.org/schema/doe-repository-metadata-profile-v1.0.json
https://repo.example.org/api/drmp/v1/repository
https://repo.example.org/api/drmp/v1/validate
https://repo.example.org/api/drmp/v1/search?q=climate+model&limit=20
```

Notes for Implementers

- Keep **field semantics** identical across MCP, OpenAPI, and JSON-LD encodings.
- Update **schema and manifest URLs** in lock-step with DRMP releases.
- Prefer UTC timestamps (ISO 8601) and UTF-8 throughout.
- Include **RDA anchor annotations** (where feasible) in validation responses to aid gap analysis.

Appendix D-2 – OpenAPI 3.1 Snippet for DMP Validation

The following OpenAPI 3.1 definition provides a minimal implementation template for repositories adopting the DRMP API specification.

The following OpenAPI 3.1 specification provides a minimal, ready-to-deploy template for repositories implementing the DOE Repository Metadata Profile (DRMP) API.
It defines standardized endpoints for retrieving and validating repository-level metadata in alignment with the DRMP JSON Schema (Appendix A) and the Model Context Protocol (MCP) interfaces (Appendix D).

Repositories may adapt this template to their own domain while preserving the field semantics, parameter names, and response structures shown below to maintain interoperability across the DOE data ecosystem.

The specification may be hosted as /openapi/repository.json or /openapi/drmp-v1.yaml and referenced in each MCP manifest to enable automated discovery and validation by AI-ready agents and external registries.

```
openapi: 3.1.0
info:
  title: DOE Repository Metadata Profile (DRMP) API
  version: 1.0.0
  description: >
    Minimal OpenAPI for DRMP-aware repositories. Exposes a repository-level
record and a validator endpoint aligned with the DRMP JSON Schema (Appendix
A).
externalDocs:
  description: DRMP JSON Schema v1.0
  url: https://repo.example.org/schema/doe-repository-metadata-profile-
v1.0.json
servers:
  - url: https://repo.example.org/api
    description: Production

paths:
  /drmp/v1/repository:
    get:
      summary: Get repository-level DRMP record
      operationId: getRepositoryRecord
      parameters:
        - in: query
          name: format
          schema:
            type: string
            enum: [json, jsonld]
          description: Preferred response serialization
      responses:
        "200":
          description: DRMP repository record
          content:
            application/json:
```

```
              schema:
                $ref: "#/components/schemas/DRMPRecord"
              examples:
                example:
                  value:
                    repositoryIdentifier: "doi:10.5439/1767341"
                    repositoryName: "ORNL Open Data Repository"
                    repositoryType: "institutional"
                    hostingInstitution: "Oak Ridge National Laboratory"
                    contactPoint: "data-curation@ornl.gov"
                    apiEndpoints:
["https://repo.example.org/api/drmp/v1/repository"]
                    metadataFormats: ["Dublin Core", "DataCite 4.6"]
                    licensePolicy:
"https://creativecommons.org/licenses/by/4.0/"
                    retentionPolicy:
"https://repo.example.org/policy/retention"
                    issued: "2024-03-01T00:00:00Z"
                    modified: "2025-09-15T00:00:00Z"
                    ontologyAlignment:
["https://purl.obolibrary.org/obo/envo.owl"]
                    preservationPolicy:
"https://repo.example.org/policy/preservation"
                    harvestPolicy: "https://repo.example.org/policy/harvest"
                    fixity:
                      premis:messageDigestAlgorithm: "SHA-256"
                      premis:messageDigest: "a2b4e6f4cce1d6..."
                    conformsTo: ["Level-2-Interoperable"]
            application/ld+json:
              schema:
                $ref: "#/components/schemas/DRMPRecord"
        "500":
          description: Server error
          content:
            application/json:
              schema: { $ref: "#/components/schemas/Error" }

  /drmp/v1/validate:
    post:
      summary: Validate a DRMP record against the DRMP JSON Schema
      operationId: validateRecord
      requestBody:
        required: true
        content:
          application/json:
            schema:
              $ref: "#/components/schemas/DRMPRecord"
      responses:
        "200":
          description: Validation result
          content:
            application/json:
              schema:
                $ref: "#/components/schemas/ValidationResult"
              examples:
                okExample:
                  value:
```

```
                    ok: true
                    level: "L2"
                    schema: "https://repo.example.org/schema/doe-repository-
metadata-profile-v1.0.json"
                    errors: []
                    deprecated: []
                    rdaSummary: { gap1: 7, gap2: 5, gap3: 0 }
                failExample:
                  value:
                    ok: false
                    level: "L1"
                    schema: "https://repo.example.org/schema/doe-repository-
metadata-profile-v1.0.json"
                    errors:
                      - path: "/licensePolicy"
                        message: "required for Level-2-Interoperable"
                      - path: "/provenance/premis:eventType"
                        message: "required when Level-3-AIReady is declared"
                    deprecated: ["/metadataFormats[text/xml]"]
                    rdaSummary: { gap1: 6, gap2: 4, gap3: 1 }
        "400":
          description: Bad request (invalid JSON)
          content:
            application/json:
              schema: { $ref: "#/components/schemas/Error" }
        "500":
          description: Server error
          content:
            application/json:
              schema: { $ref: "#/components/schemas/Error" }

components:
  securitySchemes:
    ApiKey:
      type: apiKey
      in: header
      name: X-API-Key
    OAuth2:
      type: oauth2
      flows:
        authorizationCode:
          authorizationUrl: https://auth.example.org/oauth/authorize
          tokenUrl: https://auth.example.org/oauth/token
          scopes:
            repo.read: Read repository metadata
            repo.validate: Validate repository metadata

  schemas:
    DRMPRecord:
      type: object
      additionalProperties: false
      description: DRMP v1.0 repository-level metadata (subset shown; see
Appendix A for full schema)
      properties:
        repositoryIdentifier:
          type: string
```

```
          description: Globally unique persistent identifier (DOI, ARK,
Handle)
          pattern: "^(doi|ark|hdl):"
        repositoryName:
          type: string
        repositoryType:
          type: string
        hostingInstitution:
          type: string
        contactPoint:
          type: string
          format: email
        apiEndpoints:
          type: array
          items: { type: string, format: uri }
        metadataFormats:
          type: array
          items: { type: string }
        licensePolicy:
          type: string
          format: uri
        retentionPolicy:
          type: string
          format: uri
        issued:
          type: string
          format: date-time
        modified:
          type: string
          format: date-time
        stewardshipCommitment:
          type: string
        ontologyAlignment:
          type: array
          items: { type: string, format: uri }
        themeTaxonomy:
          type: array
          items: { type: string }
        preservationPolicy:
          type: string
          format: uri
        harvestPolicy:
          type: string
          format: uri
        provenance:
          type: object
          properties:
            premis:eventIdentifier: { type: string }
            premis:eventType: { type: string }
            premis:eventDateTime: { type: string, format: date-time }
            premis:eventOutcomeInformation: { type: string }
          required: [premis:eventIdentifier, premis:eventType,
premis:eventDateTime]
        fixity:
          type: object
          properties:
            premis:messageDigestAlgorithm: { type: string }
```

```
            premis:messageDigest: { type: string }
          required: [premis:messageDigestAlgorithm, premis:messageDigest]
        conformsTo:
          type: array
          items:
            type: string
            enum: ["Level-1-MVR", "Level-2-Interoperable", "Level-3-AIReady"]
      required: ["repositoryIdentifier", "repositoryName", "conformsTo"]

    ValidationResult:
      type: object
      additionalProperties: false
      properties:
        ok: { type: boolean }
        level:
          type: string
          description: Highest conformance level satisfied by the provided
record (L1/L2/L3)
          enum: ["L1", "L2", "L3"]
        schema:
          type: string
          format: uri
        errors:
          type: array
          items:
            type: object
            properties:
              path: { type: string }
              message: { type: string }
            required: [path, message]
        deprecated:
          type: array
          items: { type: string }
        rdaSummary:
          type: object
          properties:
            gap1: { type: integer, minimum: 0 }
            gap2: { type: integer, minimum: 0 }
            gap3: { type: integer, minimum: 0 }
      required: ["ok", "level", "schema", "errors", "deprecated"]

    Error:
      type: object
      properties:
        error: { type: string, description: Machine-readable error code }
        message: { type: string, description: Human-readable explanation }
        traceId: { type: string, description: Correlation identifier for logs
}
      required: ["error", "message"]
security:
  - {}
  - ApiKey: []
  - OAuth2:
      - repo.read
      - repo.validate
```

Appendix E — DRMP Field Matrix

Legend:

- **R** = Required at that level
- **O** = Optional at that level (may be "recommended" in text)
- **—** = Not applicable / not expected at that level

Core Repository Identification & Context

Field	L1	L2	L3	Notes (repository-level)
repositoryIdentifier	R	R	R	Persistent ID for the repository (e.g., DOI, ARK).
repositoryName	R	R	R	Human-readable name.
repositoryType	R	R	R	Type/class of repository (discipline, institutional, generalist, etc.).
hostingInstitution	R	R	R	Organization(s) hosting / operating the repository.
contactPoint	R	R	R	Contact for repository support / data questions.
repositoryDescription	O	O	O	Human-readable description of the repository's scope, mission, content focus, and community served. Optional at all levels.
themeTaxonomy	O	O	O	Repository-level subject schemes/taxonomies (URIs/FAIRsharing IDs).
fundingBody	O	O	O	Funding / sponsoring organizations (names or IDs, e.g., ROR).
conformsTo	R	R	R	DRMP conformance level: "Level-1-MVR", "Level-2-Interoperable", "Level-3-AIReady".

Note: `conformsTo` stays in Appendix E (as a DRMP field) but **not** in the crosswalk matrix.

Level 2 Interoperability – Technical & Policy Signals

Field	L1	L2	L3	Notes
apiEndpoints	—	R	R	Repository APIs (URLs, base endpoints, etc.).
metadataFormats	—	R	R	Repository-level metadata formats/schemas exposed by the repository (e.g., DCAT, DataCite, Dublin Core), typically via APIs, OAI-PMH, or metadata exports. Not per-dataset metadata formats.
licensePolicy	—	R	R	URI for repository-level license / terms of use.
retentionPolicy	—	R	R	URI for retention / deletion policy.

Field	L1	L2	L3	Notes
harvestPolicy	—	R	R	URI or description of harvesting/aggregation policy.
issued	—	R	R	Creation date of the repository (or current version) metadata record.
modified	—	R	R	Last updated date for the repository metadata record.
stewardshipCommitment	—	R	R	Statement/URI describing long-term stewardship and sustainability commitments.
persistentIdentifierPolicy	—	R	R	Machine-actionable PID policy object (pidSystems + optional pidPolicy URI).
accessModel	—	O	R	Access patterns (open, restricted, controlled, etc.; repository-level default).
authorizationModel	—	O	R	How authorization is controlled (roles, groups, federated identities; repo-level default).
capabilities	—	O	R	Enumerated repository capabilities (e.g., APIs, bulk download, versioning, validation, webhooks).

Intent: at Level 2, some of these may be present; at Level 3, they become required to support AI and MCP use cases.

Level 3 AI-Readiness – Semantics, Preservation, and AI Governance

Field	L1	L2	L3	Notes
ontologyAlignment	—	—	R	Array of URIs for ontologies/vocabularies/schemas used by the repository to describe/classify holdings.
ontologyVersion	—	—	O	Array of **versioned** URIs corresponding to ontologyAlignment where available; optional but recommended.
preservationPolicy	—	—	R	URI describing repository's preservation / long-term curation policy.
provenance	—	—	R	Provenance and validation metadata for repository-level metadata generation and updates.
fixity	—	—	R	Repository-level fixity/integrity signals (checksums for key objects, metadata dumps, etc.).
aiUsePolicy	—	—	R	**Structured AI governance object** (trainingPermission, derivativeModelPermission, commercialAIUse, etc., plus aiPolicy URI).
mcpTools	—	—	R	MCP tools, models, or plug-ins exposed for repository interaction (URIs, IDs, or descriptors).
semanticContext	—	—	O	Array of JSON-LD / RDF @context URIs for machine reasoning; **strongly recommended**. Includes MCP context when implemented.

Appendix F – Repository Policy Starter Kit

This appendix provides short example policies that repositories can use as starting points when developing their own public-facing documentation. These templates illustrate the kinds of license, retention, harvest, preservation, and PID policies expected for DRMP Level 2 and Level 3 readiness.

These examples are **not official DOE or institutional policies**, and they should not be adopted verbatim. Any repository implementing or adapting this language must follow its institution's review and approval process, including legal or compliance vetting as required. The examples here are intended to spark discussion and provide practical scaffolding, not to replace local policy guidance.

Example policies provided:

- License Policy
- Retention Policy
- Harvest Policy
- Persistent Identifier Policy
- Preservation Policy
- AI Use Policy

F.1 License Policy (Example Page)

Title: *Repository License and Use Policy*
URL: https://example-repo.org/policies/license
Original Policy Date: *December 23, 2024*
Last Modified: *July 24, 2025*
Current Version: *1.1*

1. Overview
The Example Research Repository ("the Repository") makes datasets, metadata, and documentation available for public use under standardized open licenses. This policy describes how licenses are assigned and how users may reuse repository content.

2. License Assignment
Datasets are assigned a license by the depositing researcher or project.
The repository **strongly recommends** the use of:

- **Creative Commons Attribution 4.0 (CC-BY-4.0)** for general research data
- **Open Data Commons Open Database License (ODbL)** for structured databases

Metadata generated by the repository staff is released under **CC0 1.0**.

3. User Rights & Responsibilities
Users may download, reuse, and redistribute materials according to the terms of the license attached to each dataset. Users must:

- Retain attribution where required
- Note any modifications
- Comply with downstream community or journal requirements

4. Repository Responsibilities
The repository ensures that:

- License information is included in dataset records
- Machine-readable license metadata (dct:license) is exposed through APIs
- Users can filter datasets by license type

5. Contact
For questions or requests to update a license, email **support@example-repo.org**.

F-2 Retention Policy (Example Page)

Title: *Data Retention and Lifecycle Policy*
URL: https://example-repo.org/policies/retention
Original Policy Date: *December 23, 2024*
Last Modified: *July 24, 2025*
Current Version: *1.1*

1. Purpose
This policy outlines how long datasets and associated materials are retained, how long metadata is preserved, and what conditions apply to removal or deprecation.

2. Retention Timeframes
Published datasets: retained for a minimum of **10 years** after publication.
Metadata records: retained **indefinitely** to support discovery and citation.
Embargoed datasets: embargo duration is determined by the depositor (max. 3 years).

3. Removal and Deprecation
Datasets may be withdrawn when:

- Legal or privacy concerns are identified
- The depositor requests removal for justified reasons
- A dataset is superseded by a corrected or curated version

Withdrawn datasets leave a **tombstone page** containing:

- Citation
- DOI
- Reason for withdrawal
- Contact information

4. File Formats and Obsolescence
The repository monitors high-risk formats and may repackage or migrate files to ensure future accessibility.

5. Review Cycle
This policy is reviewed every **two years**.

F-3 Harvest Policy (Example Page)

Title: *Metadata Harvesting and API Access Policy*
URL: https://example-repo.org/policies/harvest
Original Policy Date: *December 23, 2024*
Last Modified: *July 24, 2025*
Current Version: *1.1*

1. Purpose
This policy defines how external systems may harvest metadata and interact with repository APIs.

2. Harvesting Standards Supported
The repository supports:

- **OAI-PMH 2.0** at https://example-repo.org/oai
- **DCAT-US v3 JSON** export at https://example-repo.org/api/dcat
- **Search API** at https://example-repo.org/api/v1/search

3. Conditions of Use
External harvesting systems must:

- Respect a maximum frequency of **one request per second**
- Include a valid User-Agent header
- Avoid high-volume full-content scraping of file downloads

4. Access Restrictions
Metadata is **openly harvestable**.
Full datasets may require authentication.
Rate limits may apply during periods of heavy system load.

5. Termination of Access
The repository may suspend hosts that violate rate limits or misuse the service.

F-4 Persistent Identifier (PID) Policy (Example Page)

Title: *Persistent Identifier Assignment and Management Policy*
URL: https://example-repo.org/policies/pids
Original Policy Date: *December 23, 2024*
Last Modified: *July 24, 2025*
Current Version: *1.1*

1. Overview
This policy describes the use of persistent identifiers (PIDs) for datasets, researchers, organizations, and projects within the repository.

2. PID Systems Supported
The repository uses the following PID systems:

- **DOI** (DataCite) for datasets, software, and documentation
- **ORCID iD** for authors
- **ROR ID** for organizations
- **Handle** for internal references and legacy records
- **LocalID** for temporary objects during ingestion workflows

3. DOI Assignment Rules
A DOI is minted when a dataset is approved for publication.
DOIs are never re-assigned or deleted.
Updated versions receive **new DOIs**, with version links exposed.

4. Responsibility and Registration
The repository is the **authoritative registration agency** for DOIs minted under its prefix. All DOIs are registered with DataCite including:

- Metadata record
- Landing page
- Version chain
- Funding references

5. PID Governance
The repository maintains:

- A public PID namespace documentation page
- A commitment to maintain DOI landing pages for at least **10 years**

F-5 Preservation Policy (Example Page)

Title: *Digital Preservation and Integrity Policy*
URL: https://example-repo.org/policies/preservation
Original Policy Date: *December 23, 2024*
Last Modified: *July 24, 2025*
Current Version: *1.1*

1. Purpose
This policy outlines the repository's long-term preservation strategy, including fixity checking, format sustainability, and migration processes.

2. Preservation Scope
The repository preserves:

- Published datasets
- Accompanying documentation
- Metadata records
- Software needed for interpretation (when possible)

3. Fixity and Integrity
Checksums (SHA-256) are generated for all files upon ingest.
Fixity is revalidated **every 90 days**.
Any mismatches trigger an automatic preservation audit.

4. Format Sustainability
Preferred formats include:

- CSV, NetCDF, HDF5, Parquet
- JSON, XML, YAML for metadata
- PNG/JPEG for images (archival TIFF when available)

5. Migration and Refresh Strategy
Files in deprecated formats may be migrated to preservation formats after curator review.
Original bitstreams are always retained unless legally restricted.

6. Storage Strategy
Data is stored across:

- A primary high-availability disk tier
- A secondary cloud replica
- A cold long-term archival tape tier

7. Policy Review
Updated every **two years**, or sooner when technology or sustainability needs change.

F-6 AI Use Policy (Example Page)

Title: *AI Use and Machine Interaction Policy*
URL (example): https://example-repo.org/policies/ai-use
Original Policy Date: *December 23, 2024*
Last Modified: *July 24, 2025*
Current Version: *1.1*

1. Overview
This policy describes how artificial intelligence (AI) systems, machine agents, and automated tools may interact with the Repository. It defines the conditions under which repository content may be used for model training, evaluation, derivative model creation, or automated access.

2. Training Permission
Unless otherwise stated in individual dataset records:

- **Metadata-only training** is permitted.
- **Full dataset training** requires approval from the depositing researcher or the Repository's data governance office.
- Datasets with restricted, sensitive, or export-controlled content may not be used for training.
- Dataset-level restrictions always override repository-wide defaults.

3. Derivative Models
Models trained using Repository metadata or datasets must:

- Cite the Repository and any relevant dataset DOIs.
- Clearly document whether training inputs included full data, metadata, or mixed sources.
- Not imply endorsement by the Repository or its host institution.

Derivative model publication may be restricted for sensitive datasets.

4. Commercial AI Use
Commercial AI use is permitted only when dataset-specific license terms allow it.
If a dataset's license prohibits commercial applications, machine agents must enforce those restrictions.

5. Model Evaluation Permission
AI models may evaluate or benchmark against Repository metadata or datasets under the following conditions:

- Evaluation tasks must not attempt to re-identify individuals from anonymized datasets.
- Benchmark logs should record dataset versions used for reproducibility.

6. Automated Agents and Interactions
Machine agents accessing the Repository (including crawlers, MCP tools, AI agents, and workflow bots) must:

- Identify themselves via User-Agent headers
- Respect documented rate limits
- Avoid scraping restricted or embargoed files

- Comply with authentication requirements where applicable
- Violations may result in temporary or permanent access restrictions.

7. AI Governance and Compliance
AI use must comply with:
- Repository license terms
- DOE public access and data management policies
- Institutional guidelines for responsible AI and automated decision-making

8. Policy Review
This policy is reviewed every two years or when changes in AI governance, privacy requirements, or funding agency guidance require updates.